Pandas

(开始时间：2019/10/21)

(by 孟智超)

1 CATLOG

1 CATLOG
2 一、pandas读取数据
2.1 pandas 设置
2.1.1 显示更多行
2.1.2 显示更多列
2.1.3 改变列宽
2.1.4 设置float列的精度
2.1.5 数字格式化显示
2.1.6 更改绘图方法
2.1.7 配置info()的输出
2.1.8 打印出当前设置并重置所有选项
2.2 读取纯文本文件
2.2.1 `pd.read_csv`
2.2.2 读取txt文件
2.3 读取excel文件
2.4 读取MySQL数据库
3 二、pandas 数据结构
3.1 Series
3.1.1 列表创建
3.1.2 创建标签索引
3.1.3 字典创建
3.1.4 根据标签索引查询数据
3.2 DataFrame
3.2.1 多个字典创建
3.3 从DataFrame中查询出Series
3.3.1 查询多列
3.3.2 查询一行
3.3.3 查询多行
4 三、Pandas查询数据
4.1 读取数据
4.2 查询数据
4.2.1 单个label
4.2.2 批量查询
4.2.3 范围查询
4.2.4 使用条件表达式查询
4.2.5 调用函数查询
5 四、Pandas新增数据列
5.1 直接赋值
5.2 df.apply方法
5.3 df.assign方法
5.4 按条件选择分组分别赋值
6 五、Pandas数据统计函数
6.1 汇总类统计

6.2 唯一去重和按值计数
6.2.1 唯一性去重
6.2.2 按值计数
6.3 相关系数和协方差
6.3.1 协方差矩阵
6.3.2 相关系数矩阵
6.3.3 单独查看某两个特征的相关系数
7 六、Pandas对缺失值的处理
7.1 实例：特殊Excel的读取、清洗、处理
7.1.1 步骤1：读取excel的时候，忽略前几个空行
7.1.2 步骤2：检测空值
7.1.3 步骤3：删除掉全是空值的列
7.1.4 步骤4：删除掉全是空值的行
7.1.5 步骤5：将分数列为空的填充为0分
7.1.6 步骤6：将姓名的缺失值填充
8 七、Pandas的SettingWithCopyWarning报警
8.1 读取数据
8.2 复现
8.3 原因
8.4 解决方法1
8.5 解决方法2
9 八、pandas数据排序
9.1 Series的排序
9.2 DataFrame的排序
9.2.1 单列排序
9.2.2 多列排序
10 九、Pandas字符串处理
10.1 Series的str属性
10.2 字符串条件查询
10.3 需要多次str处理的链式操作
10.4 正则表达式
11 十、Pandas的axis参数怎么理解?
11.1 单列drop
11.2 单行drop
11.3 按axis=0、index执行mean聚合操作
11.4 按axis=1、columns执行mean聚合操作
11.5 再次举例，加深理解
12 十一、Pandas索引作用和层次化索引
12.1 索引作用
12.1.1 查询数据
12.1.2 提升查询性能
12.1.3 实验1：完全随机的顺序查询
12.1.4 实验2：将index排序后的查询
12.1.5 自动对齐数据
12.1.6 更多更强大的数据结构支持
12.2 层次化索引
12.2.1 层次化索引（hierarchical indexing）

12.2.2 重排与分级排序（Reordering and Sorting Levels）
12.2.3 根据级别汇总统计（Summary Statistics by Level）
12.2.4 使用DataFrame的列进行索引(Indexing with a DataFrame's columns)
13 十二、Pandas怎样实现DataFrame的Merge
13.1 merge的语法：
13.2 电影数据集的join实例
13.2.1 电影评分数据集
13.2.2 理解merge时数量的对齐关系
13.2.3 理解join方法的区别
13.2.4 非Key字段重名
13.3 索引上的合并（Merging on Index）
14 十三、Pandas实现数据的合并concat
14.1 pandas.concat
14.1.1 默认
14.1.2 参数指定
14.1.3 轴向连接（Concatenating Along an Axis)
14.2 DataFrame.append
14.2.1 给1个dataframe添加另一个dataframe
14.2.2 一行一行的给DataFrame添加数据
15 十四、Pandas重塑和轴向旋转
15.1 重塑层次化索引
15.2 将“长格式”旋转为“宽格式”
15.3 将“宽格式”旋转为“长格式”
16 十五、Pandas怎样实现groupby分组统计
16.1 分组使用聚合函数做数据统计
16.1.1 单个列groupby
16.1.2 多个列groupby
16.1.3 同时查看多种数据统计
16.1.4 查看单列的结果数据统计
16.1.5 不同列使用不同的聚合函数
16.2 遍历groupby的结果理解执行流程
16.2.1 遍历单个列聚合的分组
16.2.2 遍历多个列聚合的分组
16.3 实例分组探索天气数据
16.3.1 查看每个月的最高温度
16.3.2 查看每个月的最高温度、最低温度、平均空气质量指数
17 十六、Pandas_map、apply、applymap
17.1 map用于Series值的转换
17.1.1 方法1：Series.map(dict)
17.1.2 方法2：Series.map(function)
17.2 apply用于Series和DataFrame的转换
17.2.1 Series.apply(function)
17.2.2 DataFrame.apply(function)
17.3 applymap用于DataFrame所有值的转换
18 十七、Pandas怎样对每个分组应用apply函数?
18.1 实例1：怎样对数值列按分组的归一化?
18.1.1 演示：用户对电影评分的归一化

18.2 实例2：怎样取每个分组的TOPN数据?
19 十八、pandas 画图
19.1 基础图表
19.1.1 柱状图
19.1.2 柱状图堆叠
19.1.3 堆叠柱状图
19.1.4 面积图
19.1.5 堆叠面积图
19.1.6 密度估计图
19.1.7 直方图
19.1.8 箱线图
19.1.9 折线图
19.1.10 散点图
19.1.11 六边形图
19.2 多子图
20 十九、pandas函数补充
20.1 pd.DataFrame.cumsum()

2 一、pandas读取数据

- 导入需要的包

```
import pandas as pd
import numpy as np
import warnings
import os

warnings.filterwarnings('ignore')
```

2.1 pandas 设置

- 通过 `pandas` 的使用，我们经常要交互式地展示表格（`dataframe`）、分析表格。而表格的格式就显得尤为重要了，因为大部分时候如果我们直接展示表格，格式并不是很友好。

  其实呢，这些痛点都可以通过 `pandas` 的 `option` 来解决。短短几行代码，只要提前配置好，一次设置好，全局生效，perfect!

- 使用方法和属性

```
# 使用方法
import pandas as pd
pd.set_option()
pd.get_option()

# 使用属性，例如展示的最大行数
pd.option.display.max_rows
```

- 常用设置汇总

```
pd.set_option('display.max_rows',xxx) # 最大行数
pd.set_option('display.min_rows',xxx) # 最小显示行数
pd.set_option('display.max_columns',xxx) # 最大显示列数
pd.set_option ('display.max_colwidth',xxx) #最大列字符数
pd.set_option( 'display.precision',2) # 浮点型精度
pd.set_option('display.float_format','{:,}'.format) #逗号分隔数字
pd.set_option('display.float_format',  '{:,.2f}'.format) #设置浮点精度
pd.set_option('display.float_format', '{:.2f}%'.format) #百分号格式化
pd.set_option('plotting.backend', 'altair') # 更改后端绘图方式
pd.set_option('display.max_info_columns', 200) # info输出最大列数
pd.set_option('display.max_info_rows', 5) # info计数null时的阈值
pd.describe_option() #展示所有设置和描述
pd.reset_option('all') #重置所有设置选项
```

2.1.1 显示更多行

- 默认情况下，`pandas` 是不超出屏幕的显示范围的，如果表的行数很多，它会截断中间的行只显示一部分。我们可以通过设置 `display.max_rows` 来控制显示的最大行数，比如我想设置显示200行。

```
pd.set_option('display.max_rows', 200)
# pd.options.display.max_rows = 200
```

  如果行数超过了 `display.max_rows`，那么 `display.min_rows` 将确定显示的部分有多少行。因为 `display.min_rows` 的默认行数为5，,下面例子只显示前5行和最后5行，中间的所有行省略。

  同理，也可根据自己的习惯显示可显示的行数，比如10, 20..

```
pd.set_option('display.min_rows', 10)
# pd.options.display.min_rows = 10
```

  还可以直接重置

```
# 重置
pd.reset_option('display.max_rows')
```

- 查看当前最大显示行数

```
# pd.get_option('display.max_columns')
pd.options.display.max_columns


"""输出如下"""
60
```

2.1.2 显示更多列

- 行可以设置，同样的列也可以设置，`display.max_columns` 控制着可显示的列数，默认值为20。

```
pd.set_option('display.max_columns', 200)
# pd.options.display.max_columns = 200
```

  重置

```
# 重置
pd.reset_option('display.max_columns')
```

- 查看当前最大显示列数

```
pd.get_option('display.max_columns')
# pd.options.display.max_columns


"""输出如下"""
20
```

2.1.3 改变列宽

- `pandas` 对列中显示的字符数有一些限制，默认值为50字符。所以，有的值字符过长就会显示省略号。如果想全部显示，可以设置 `display.max_colwidth`，比如设置成500。

```
pd.set_option ('display.max_colwidth',500)
# pd.options.display.max_colwidth = 500
```

重置

```
pd.reset_option('display.max_colwidth')
```

- 查看当前列宽

```
# pd.get_option ('display.max_colwidth')
pd.options.display.max_colwidth


"""输出如下"""
500
```

2.1.4 设置float列的精度

- 对于 `float` 浮点型数据， `pandas` 默认情况下只显示小数点后6位。我们可以通过预先设置 `display.precision` 让其只显示2位，避免后面重复操作。

```
# pd.set_option( 'display.precision',2)
pd.options.display.precision = 2
```

重置

```
pd.reset_option('display.precision')
```

- 查看当前 `float` 列精度(默认为6)

```
# pd.get_option( 'display.precision')
pd.options.display.precision

"""输出如下"""
6
```

2.1.5 数字格式化显示

- `pandas` 中有一个选项 `display.float_formatoption` 可以用来格式化任何浮点列。这个仅适用于浮点列，对于其他数据类型，必须将它们转换为浮点数才可以。

2.1.5.1 用逗号格式化大值数字

- 例如 1200000 这样的大数字看起来很不方便，所以我们用逗号进行分隔。

```
pd.set_option('display.float_format','{:,}'.format)
```

2.1.5.2 设置数字精度

- 和上面 `display.precision` 有点类似，假如我们只关心小数点后的2位数字，我们可以这样设置格式化：

```
pd.set_option('display.float_format',  '{:,.2f}'.format)
```

2.1.5.3 百分号格式化

- 如果我们要显示一个百分比的列，可以这样设置。

```
pd.set_option('display.float_format', '{:.2f}%'.format)
```

  或者其它币种的符号等均可，只需要在大括号 `{}` 前后添加即可。

2.1.6 更改绘图方法

- 默认情况下， `pandas` 使用 `matplotlib` 作为绘图后端。从 0.25 版本开始， `pandas` 提供了使用不同后端选择，比如 `plotly` ， `bokeh` 等第三方库，但前提是你需要先安装起来。

  设置很简单，只要安装好三方库后，同样只需要一行。

```
import pandas as pd
import numpy as np
pd.set_option('plotting.backend', 'altair')
data = pd.Series(np.random.randn(100).cumsum())
data.plot()
```

2.1.7 配置info()的输出

- `pandas` 中我们经常要使用 `info()` 来快速查看 `DataFrame` 的数据情况。但是， `info` 这个方法对要分析的最大列数是有默认限制的，并且如果数据集中有 `null` ，那么在大数据集计数统计时会非常慢。

  `pandas` 提供了两种选择。

2.1.7.1 设置要分析的最大列数

- 设置要分析的最大列数，默认为100

  比如，在分析有 150 个特征的数据集时，我们可以设置 `display.max_info_columns` 为涵盖所有列的值，比如将其设置为 200：

```
pd.set_option('display.max_info_columns', 200)
display.max_info_columns = 200
```

2.1.7.2 设置计数null时的阈值

- 设置计数null时的阈值,默认为1690785

  在分析大型数据集时，`df.info()` 由于要计算所有 `null`，导致速度很慢。因此我们可以简单地设置 `display.max_info_rows` 为一个小的值来避免计数，例如只在行数不超过5时才计数 `null`：

```
pd.set_option('display.max_info_rows', 5)
display.max_info_rows = 5
```

2.1.8 打印出当前设置并重置所有选项

2.1.8.1 打印出当前设置

- 打印出当前设置

  `pd.describe_option()` 将打印出设置的描述及其当前值。

```
pd.describe_option()
```

- 还可以打印特定的选项，例如，行显示。

```
# 具体的搜索
pd.describe_option('rows')
```

2.1.8.2 重置

- 我们还可以直接全部重置

```
pd.reset_option('all')
```

2.2 读取纯文本文件

2.2.1 `pd.read_csv`

- 使用默认的标题行、逗号分隔符

```
import pandas as pd
fpath = "./datas/ml-latest-small/ratings.csv"
'''使用pd.read_csv读取数据'''
ratings = pd.read_csv(fpath)
''' 查看前几行数据'''
ratings.head()
```

	userId	movieId	rating	timestamp
0	1	1	4.0	964982703
1	1	3	4.0	964981247
2	1	6	4.0	964982224
3	1	47	5.0	964983815
4	1	50	5.0	964982931

查看数据的形状

```
'''查看数据的形状，返回(行数、列数)'''
ratings.shape
(100836, 4)

"""输出如下"""
Index(['userId', 'movieId', 'rating', 'timestamp'], dtype='object')
```

查看索引列

```
'''查看索引列'''
ratings.index

"""输出如下"""
RangeIndex(start=0, stop=100836, step=1)
```

查看每列的数据类型

```
'''查看每列的数据类型'''
ratings.dtypes

"""输出如下"""
userId         int64
movieId        int64
rating       float64
timestamp      int64
dtype: object
```

2.2.2 读取txt文件

- 自己指定分隔符、列名

```
fpath = "./datas/crazyant/access_pvuv.txt"

pvuv = pd.read_csv(
    fpath,
    sep="\t", # 指定分隔符
    header=None, # 告诉pandas该数据没有 特征名称（列名）
    names=['pdate', 'pv', 'uv']  # 自己指定特征名称（列名）
)
```

```
pvuv
```

	pdate	pv	uv
0	2019-09-10	139	92
1	2019-09-09	185	153
2	2019-09-08	123	59
3	2019-09-07	65	40
4	2019-09-06	157	98
5	2019-09-05	205	151
6	2019-09-04	196	167
7	2019-09-03	216	176
8	2019-09-02	227	148
9	2019-09-01	105	61

2.3 读取excel文件

- `read_excel`

```
fpath = "./datas/crazyant/access_pvuv.xlsx"
pvuv = pd.read_excel(fpath)
```

```
pvuv
```

	pdate	pv	uv
0	2019-09-10	139	92
1	2019-09-09	185	153
2	2019-09-08	123	59
3	2019-09-07	65	40
4	2019-09-06	157	98
5	2019-09-05	205	151
6	2019-09-04	196	167
7	2019-09-03	216	176
8	2019-09-02	227	148
9	2019-09-01	105	61

2.4 读取MySQL数据库

- `pymysql`

```python
import pymysql
conn = pymysql.connect(
        host='127.0.0.1',
        user='root',
        password='123qwe',
        database='test',
        charset='utf8'
    )

mysql_page = pd.read_sql("select * from crazyant_pvuv", con=conn)
mysql_page
```

	pdate	pv	uv
0	2019-09-10	139	92
1	2019-09-09	185	153
2	2019-09-08	123	59
3	2019-09-07	65	40
4	2019-09-06	157	98
5	2019-09-05	205	151
6	2019-09-04	196	167
7	2019-09-03	216	176
8	2019-09-02	227	148
9	2019-09-01	105	61

3 二、pandas 数据结构

3.1 Series

- Series是一种类似于一维数组的对象，它由一组数据（不同数据类型）以及一组与之相关的数据标签（即索引）组成。

3.1.1 列表创建

- 仅有数据列表即可产生最简单的Series

```
s1 = pd.Series([1,'a',5.2,7])
'''左侧为索引，右侧是数据'''
s1

"""输出如下"""
0       1
1       a
2     5.2
3       7
dtype: object
```

获取索引

```
'''获取索引'''
s1.index

"""输出如下"""
RangeIndex(start=0, stop=4, step=1)
```

获取数据

```
''' 获取数据'''
s1.values

"""输出如下"""
array([1, 'a', 5.2, 7], dtype=object)
```

3.1.2 创建标签索引

- 创建一个具有标签索引的Series

```
s2 = pd.Series([1, 'a', 5.2, 7], index=['d','b','a','c'])
s2

"""输出如下"""
d       1
b       a
a     5.2
c       7
dtype: object
```

获取索引：

```
s2.index

"""输出如下"""
Index(['d', 'b', 'a', 'c'], dtype='object')
```

3.1.3 字典创建

- 使用Python字典创建Series

```
sdata={'Ohio':35000,'Texas':72000,'Oregon':16000,'Utah':5000}
s3=pd.Series(sdata)
s3

"""输出如下"""
Ohio      35000
Texas     72000
Oregon    16000
Utah       5000
dtype: int64
```

3.1.4 根据标签索引查询数据

- 类似Python的字典dict

```
s2

"""输出如下"""
d       1
b       a
a     5.2
c       7
dtype: object
```

```
s2['a']

"""输出如下"""
5.2
```

```
type(s2['a'])

"""输出如下"""
float
```

```
s2[['b','a']]

"""输出如下"""
b       a
a     5.2
dtype: object
```

```
type(s2[['b','a']])

"""输出如下"""
pandas.core.series.Series
```

3.2 DataFrame

- `DataFrame` 是一个表格型的数据结构
    1. 每列可以是不同的值类型（数值、字符串、布尔值等）
    2. 既有行索引index,也有列索引columns
    3. 可以被看做由Series组成的字典

创建dataframe最常用的方法，见02节读取纯文本文件、excel、mysql数据库

3.2.1 多个字典创建

- 根据多个字典序列创建dataframe

```
data={
        'state':['Ohio','Ohio','Ohio','Nevada','Nevada'],
        'year':[2000,2001,2002,2001,2002],
        'pop':[1.5,1.7,3.6,2.4,2.9]
    }
df = pd.DataFrame(data)
df
```

	state	year	pop
0	Ohio	2000	1.5
1	Ohio	2001	1.7
2	Ohio	2002	3.6
3	Nevada	2001	2.4
4	Nevada	2002	2.9

每一列的数据类型

```
# 每一列的数据类型
df.dtypes

"""输出如下"""
state      object
year        int64
pop       float64
dtype: object
```

查看所有的列名

```
# 查看所有的列名（特征名称、或者理解为列的索引）
df.columns
"""输出如下"""
Index(['state', 'year', 'pop'], dtype='object')
```

查看行索引

```
# 查看行索引
df.index
"""输出如下"""
RangeIndex(start=0, stop=5, step=1
```

3.3 从DataFrame中查询出Series

- 从DataFrame中查询出Series
    1. 如果只查询一行、一列，返回的是pd.Series
    2. 如果查询多行、多列，返回的是pd.DataFrame

```
df
```

	state	year	pop
0	Ohio	2000	1.5
1	Ohio	2001	1.7
2	Ohio	2002	3.6
3	Nevada	2001	2.4
4	Nevada	2002	2.9

3.3.1 查询多列

- 查询多列，结果是一个pd.DataFrame

```
df[['year', 'pop']]
"""输出如下"""
```

	year	pop
0	2000	1.5
1	2001	1.7
2	2002	3.6
3	2001	2.4
4	2002	2.9

```
type(df[['year', 'pop']])

"""输出如下"""
pandas.core.frame.DataFram
```

3.3.2 查询一行

- 查询一行，结果是一个 `pd.Series`

```
df.loc[1]

"""输出如下"""
state    Ohio
year     2001
pop       1.7
Name: 1, dtype: object
```

```
type(df.loc[1])

"""输出如下"""
pandas.core.series.Series
```

3.3.3 查询多行

- 查询多行，结果是一个 `pd.DataFrame`

  查询第2-4行

```
df.loc[1:3]

"""输出如下"""
```

	state	year	pop
1	Ohio	2001	1.7
2	Ohio	2002	3.6
3	Nevada	2001	2.4

```
type(df.loc[1:3])

"""输出如下"""
pandas.core.frame.DataFrame
```

4 三、Pandas查询数据

- Pandas查询数据的几种方法
    1. `df.loc` 方法，根据行、列的标签值查询
    2. `df.iloc` 方法，根据行、列的数字位置查询
    3. `df.where` 方法
    4. `df.query` 方法

  `.loc` 既能查询，又能覆盖写入，强烈推荐！
- `Pandas` 使用 `df.loc` 查询数据的方法
    1. 使用单个label值查询数据
    2. 使用值列表批量查询
    3. 使用数值区间进行范围查询
    4. 使用条件表达式查询
    5. 调用函数查询
- 注意
    1. 以上查询方法，既适用于行，也适用于列
    2. 注意观察降维 `dataFrame` > `Series` >值

```
print(pd.__version__)

 1.1.5
```

4.1 读取数据

- 数据为北京2018年全年天气预报

```
df = pd.read_csv("./datas/beijing_tianqi/beijing_tianqi_2018.csv")
df.head()
```

	ymd	bWendu	yWendu	tianqi	fengxiang	fengli	aqi	aqiInfo	aqiLevel
0	2018-01-01	3°C	-6°C	晴~多云	东北风	1-2级	59	良	2
1	2018-01-02	2°C	-5°C	阴~多云	东北风	1-2级	49	优	1
2	2018-01-03	2°C	-5°C	多云	北风	1-2级	28	优	1
3	2018-01-04	0°C	-8°C	阴	东北风	1-2级	28	优	1
4	2018-01-05	3°C	-6°C	多云~晴	西北风	1-2级	50	优	1

设定索引为日期

```python
# 设定索引为日期，方便按日期筛选
df.set_index('ymd', inplace=True)
```

```python
# 时间序列见后续，本次按字符串处理
df.index

"""输出如下"""
Index(['2018-01-01', '2018-01-02', '2018-01-03', '2018-01-04', '2018-01-05',
       '2018-01-06', '2018-01-07', '2018-01-08', '2018-01-09', '2018-01-10',
       ...
       '2018-12-22', '2018-12-23', '2018-12-24', '2018-12-25', '2018-12-26',
       '2018-12-27', '2018-12-28', '2018-12-29', '2018-12-30', '2018-12-31'],
      dtype='object', name='ymd', length=365)
```

替换掉温度的后缀

```python
# 替换掉温度的后缀℃
df.loc[:,'bWendu'] = df['bWendu'].str.replace('℃','').astype('int32')
df.loc[:,'yWendu'] = df['yWendu'].str.replace('℃','').astype('int32')

df.dtypes

"""输出如下"""
bWendu        int32
yWendu        int32
tianqi       object
fengxiang    object
fengli       object
aqi           int64
aqiInfo      object
aqiLevel      int64
dtype: object
```

```python
df.head()
"""输出如下"""
```

ymd	bWendu	yWendu	tianqi	fengxiang	fengli	aqi	aqiInfo	aqiLevel
2018-01-01	3	-6	晴~多云	东北风	1-2级	59	良	2
2018-01-02	2	-5	阴~多云	东北风	1-2级	49	优	1
2018-01-03	2	-5	多云	北风	1-2级	28	优	1
2018-01-04	0	-8	阴	东北风	1-2级	28	优	1
2018-01-05	3	-6	多云~晴	西北风	1-2级	50	优	1

4.2 查询数据

4.2.1 单个label

- 使用单个label值查询数据
- 行或者列，都可以只传入单个值，实现精确匹配

得到单个值 （2018年1月3日的最高温度）

```
# 得到单个值 （2018年1月3日的最高温度）
df.loc['2018-01-03', 'bWendu']

"""输出如下"""
2
```

得到一个Series（2018年1月3日的最高温度和最低温度）

```
# 得到一个Series（2018年1月3日的最高温度和最低温度）
df.loc['2018-01-03',['bWendu','yWendu']]

"""输出如下"""
bWendu      2
yWendu     -5
Name: 2018-01-03, dtype: object
```

4.2.2 批量查询

- 使用值列表批量查询

得到Series：18年1月3，4，5日的最高温度

```
# 得到Series
df.loc[['2018-01-03','2018-01-04','2018-01-05'], 'bWendu']

"""输出如下"""
ymd
2018-01-03    2
2018-01-04    0
2018-01-05    3
Name: bWendu, dtype: int32
```

得到DataFrame：18年1月3，4，5日的最高温度和最低温度

```
# 得到DataFrame
df.loc[['2018-01-03','2018-01-04','2018-01-05'], ['bWendu',
'yWendu']]

"""输出如下"""
```

	bWendu	yWendu
ymd		
2018-01-03	2	-5
2018-01-04	0	-8
2018-01-05	3	-6

4.2.3 范围查询

- 使用数值区间进行范围查询

  注意：区间既包含开始，也包含结束

  行index按区间 （2018年1月3日至5号的最高温度和最低温度）

```
# 行index按区间 （2018年1月3日至5号的最高温度和最低温度） （series）
df.loc['2018-01-03':'2018-01-05', 'bWendu']

"""输出如下"""
ymd
2018-01-03    2
2018-01-04    0
2018-01-05    3
Name: bWendu, dtype: int32
```

列index按区间

```
# 列index按区间（series）
df.loc['2018-01-03', 'bWendu':'fengxiang']

"""输出如下"""
bWendu         2
yWendu        -5
tianqi        多云
fengxiang     北风
Name: 2018-01-03, dtype: object
```

行和列都按区间查询

```
# 行和列都按区间查询(DataFrame)
df.loc['2018-01-03':'2018-01-05', 'bWendu':'fengxiang']

"""输出如下"""


```

ymd	bWendu	yWendu	tianqi	fengxiang
2018-01-03	2	-5	多云	北风
2018-01-04	0	-8	阴	东北风
2018-01-05	3	-6	多云~晴	西北风

4.2.4 使用条件表达式查询

- bool列表的长度得等于行数或者列数

4.2.4.1 简单条件查询

- 简单条件查询，最低温度低于-10度的列表

```
df.loc[df['yWendu']<-10,:]

"""输出如下"""
```

ymd	bWendu	yWendu	tianqi	fengxiang	fengli	aqi	aqiInfo	aqiLevel
2018-01-23	-4	-12	晴	西北风	3-4级	31	优	1
2018-01-24	-4	-11	晴	西南风	1-2级	34	优	1
2018-01-25	-3	-11	多云	东北风	1-2级	27	优	1
2018-12-26	-2	-11	晴~多云	东北风	2级	26	优	1
2018-12-27	-5	-12	多云~晴	西北风	3级	48	优	1
2018-12-28	-3	-11	晴	西北风	3级	40	优	1
2018-12-29	-3	-12	晴	西北风	2级	29	优	1
2018-12-30	-2	-11	晴~多云	东北风	1级	31	优	1

观察一下这里的boolean条件

```
# 观察一下这里的boolean条件
df['yWendu']<-10

"""输出如下"""

ymd
2018-01-01    False
2018-01-02    False
2018-01-03    False
2018-01-04    False
2018-01-05    False
              ...
2018-12-27     True
2018-12-28     True
2018-12-29     True
2018-12-30     True
2018-12-31    False
Name: yWendu, Length: 365, dtype: bool
```

4.2.4.2 复杂条件查询

- 复杂条件查询，查一下完美天气

  注意，组合条件用&符号合并，每个条件判断都得带括号

  查询最高温度小于30度，并且最低温度大于15度，并且是晴天，并且天气为优的数据

```
## 查询最高温度小于30度，并且最低温度大于15度，并且是晴天，并且天气为优的数据
df.loc[(df['bWendu']<=30) & (df['yWendu']>15)&(df["tianqi"]=='晴') & (df["aqiLevel"]==1),:]

"""输出如下"""
```

ymd	bWendu	yWendu	tianqi	fengxiang	fengli	aqi	aqiInfo	aqiLevel
2018-08-24	30	20	晴	北风	1-2级	40	优	1
2018-09-07	27	16	晴	西北风	3-4级	22	优	1

再次观察这里的boolean条件

```
# 再次观察这里的boolean条件
(df['bWendu']<=30) & (df['yWendu']>15)&(df["tianqi"]=='晴') & (df["aqiLevel"]==1)

"""输出如下"""
ymd
2018-01-01    False
2018-01-02    False
2018-01-03    False
2018-01-04    False
2018-01-05    False
              ...
2018-12-27    False
2018-12-28    False
2018-12-29    False
2018-12-30    False
2018-12-31    False
Length: 365, dtype: bool
```

4.2.5 调用函数查询

4.2.5.1 lambda表达式

- 直接写lambda表达式

```
# 直接写lambda表达式
df.loc[lambda df : (df["bWendu"]<=30) & (df["yWendu"]>=15), :]

"""输出如下"""
```

ymd	bWendu	yWendu	tianqi	fengxiang	fengli	aqi	aqiInfo	aqiLevel
2018-04-28	27	17	晴	西南风	3-4级	125	轻度污染	3
2018-04-29	30	16	多云	南风	3-4级	193	中度污染	4
2018-05-04	27	16	晴~多云	西南风	1-2级	86	良	2
2018-05-09	29	17	晴~多云	西南风	3-4级	79	良	2
2018-05-10	26	18	多云	南风	3-4级	118	轻度污染	3
...	...	...	...	...	...	...	...	...
2018-09-15	26	15	多云	北风	3-4级	42	优	1
2018-09-17	27	17	多云~阴	北风	1-2级	37	优	1
2018-09-18	25	17	阴~多云	西南风	1-2级	50	优	1
2018-09-19	26	17	多云	南风	1-2级	52	良	2
2018-09-20	27	16	多云	西南风	1-2级	63	良	2

4.2.5.2 编写函数

- 自己编写函数

```
# 编写自己的函数，查询9月份，空气质量好的数据
def query_my_data(df):
    return df.index.str.startswith("2018-9") & (df["aqiLevel"]==1)

df.loc[query_my_data, :]

"""输出如下"""
```

ymd	bWendu	yWendu	tianqi	fengxiang	fengli	aqi	aqiInfo	aqiLevel
2018-09-01	27	19	阴~小雨	南风	1-2级	50	优	1
2018-09-04	31	18	晴	西南风	3-4级	24	优	1
2018-09-05	31	19	晴~多云	西南风	3-4级	34	优	1
2018-09-06	27	18	多云~晴	西北风	4-5级	37	优	1
2018-09-07	27	16	晴	西北风	3-4级	22	优	1
2018-09-08	27	15	多云~晴	北风	1-2级	28	优	1
2018-09-15	26	15	多云	北风	3-4级	42	优	1
2018-09-16	25	14	多云~晴	北风	1-2级	29	优	1
2018-09-17	27	17	多云~阴	北风	1-2级	37	优	1
2018-09-18	25	17	阴~多云	西南风	1-2级	50	优	1
2018-09-21	25	14	晴	西北风	3-4级	50	优	1
2018-09-22	24	13	晴	西北风	3-4级	28	优	1
2018-09-23	23	12	晴	西北风	4-5级	28	优	1
2018-09-24	23	11	晴	北风	1-2级	28	优	1
2018-09-25	24	12	晴~多云	南风	1-2级	44	优	1
2018-09-29	22	11	晴	北风	3-4级	21	优	1
2018-09-30	19	13	多云	西北风	4-5级	22	优	1

5 四、Pandas新增数据列

- 在进行数据分析时，经常需要按照一定条件创建新的数据列，然后进行进一步分析。
    1. 直接赋值
    2. df.apply方法
    3. df.assign方法
    4. 按条件选择分组分别赋值

先读取数据：

```
fpath = "./datas/beijing_tianqi/beijing_tianqi_2018.csv"
df = pd.read_csv(fpath)
```

```
# 替换掉温度的后缀℃ ,清理温度列，变成数字类型
df.loc[:,'bWendu'] = df['bWendu'].str.replace('℃','').astype('int32')
df.loc[:,'yWendu'] = df['yWendu'].str.replace('℃','').astype('int32')
```

```
df.dtypes

"""输出如下"""
ymd           object
bWendu         int32
yWendu         int32
tianqi        object
fengxiang     object
fengli        object
aqi            int64
aqiInfo       object
aqiLevel       int64
dtype: object
```

```
df.head()
```

	ymd	bWendu	yWendu	tianqi	fengxiang	fengli	aqi	aqiInfo	aqiLevel
0	2018-01-01	3	-6	晴~多云	东北风	1-2级	59	良	2
1	2018-01-02	2	-5	阴~多云	东北风	1-2级	49	优	1
2	2018-01-03	2	-5	多云	北风	1-2级	28	优	1
3	2018-01-04	0	-8	阴	东北风	1-2级	28	优	1
4	2018-01-05	3	-6	多云~晴	西北风	1-2级	50	优	1

5.1 直接赋值

- 直接赋值的方法

```
# 注意，df["bWendu"]其实是一个Series，后面的减法返回的是Series
df.loc[:,'wencha_1'] =   df['bWendu'] - df['yWendu']
df['wencha_2'] = df['bWendu'] - df['yWendu']
```

5.2 df.apply方法

- Apply a function along an axis of the DataFrame.

  Objects passed to the function are Series objects whose index is either the DataFrame's index (axis=0) or the DataFrame's columns (axis=1).

- 实例：添加一列温度类型：

  1. 如果最高温度大于33度就是高温
  2. 低于-10度是低温
  3. 否则是常温

```
def get_wendu_type(x):
    if x["bWendu"] > 33:
        return '高温'
    if x["yWendu"] < -10:
        return '低温'
    return '常温'


# 注意需要设置axis==1，这是series的index是columns
df.loc[:, "wendu_type_1"] = df.apply(get_wendu_type, axis=1)
df['wendu_type_2'] = df.apply(get_wendu_type,axis=1)

df.head()
```

	ymd	bWendu	yWendu	tianqi	fengxiang	fengli	aqi	aqiInfo	aqiLevel	wencha	wendu_type_1	wendu_type_2
0	2018-01-01	3	-6	晴~多云	东北风	1-2级	59	良	2	9	常温	常温
1	2018-01-02	2	-5	阴~多云	东北风	1-2级	49	优	1	7	常温	常温
2	2018-01-03	2	-5	多云	北风	1-2级	28	优	1	7	常温	常温
3	2018-01-04	0	-8	阴	东北风	1-2级	28	优	1	8	常温	常温
4	2018-01-05	3	-6	多云~晴	西北风	1-2级	50	优	1	9	常温	常温

查看温度类型的计数

```python
# 查看温度类型的计数
df['wendu_type_1'].value_counts()

"""输出如下"""
常温     328
高温      29
低温       8
Name: wendu_type_1, dtype: int64
```

```python
df['wendu_type_2'].value_counts()
"""输出如下"""
常温     328
高温      29
低温       8
Name: wendu_type_2, dtype: int64
```

5.3 df.assign方法

- Assign new columns to a DataFrame.

  Returns a new object with all original columns in addition to new ones.

- 实例，将摄氏度转换为华氏度

```python
# 可以同时添加多个新的列
df.assign(
    yWendu_huashi = lambda x : x["yWendu"] * 9 / 5 + 32,
    # 摄氏度转华氏度
    bWendu_huashi = lambda x : x["bWendu"] * 9 / 5 + 32
)

"""输出如下"""
```

	ymd	bWendu	yWendu	tianqi	fengxiang	fengli	aqi	aqiInfo	aqiLevel	wencha	wendu_type_1	wendu_type_2	yWendu_huashi	bWendu_huashi
0	2018-01-01	3	-6	晴~多云	东北风	1-2级	59	良	2	9	常温	常温	21.2	37.4
1	2018-01-02	2	-5	阴~多云	东北风	1-2级	49	优	1	7	常温	常温	23.0	35.6
2	2018-01-03	2	-5	多云	北风	1-2级	28	优	1	7	常温	常温	23.0	35.6
3	2018-01-04	0	-8	阴	东北风	1-2级	28	优	1	8	常温	常温	17.6	32.0
4	2018-01-05	3	-6	多云~晴	西北风	1-2级	50	优	1	9	常温	常温	21.2	37.4
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
360	2018-12-27	-5	-12	多云~晴	西北风	3级	48	优	1	7	低温	低温	10.4	23.0
361	2018-12-28	-3	-11	晴	西北风	3级	40	优	1	8	低温	低温	12.2	26.6
362	2018-12-29	-3	-12	晴	西北风	2级	29	优	1	9	低温	低温	10.4	26.6
363	2018-12-30	-2	-11	晴~多云	东北风	1级	31	优	1	9	低温	低温	12.2	28.4
364	2018-12-31	-2	-10	多云	东北风	1级	56	良	2	8	常温	常温	14.0	28.4

5.4 按条件选择分组分别赋值

- 按条件先选择数据，然后对这部分数据赋值新列
- 实例：高低温差大于10度，则认为温差大

```
# 先创建空列（这是第一种创建新列的方法）
df['wencha_type_1'] = ''

df.loc[df["bWendu"]-df["yWendu"]>10, "wencha_type_1"] = "温差大"

df.loc[df["bWendu"]-df["yWendu"]<=10, "wencha_type_1"] = "温差正常"
```

```
df["wencha_type_1"].value_counts()

"""输出如下"""
温差正常    187
温差大     178
Name: wencha_type_1, dtype: int64
```

- 实例等同于 `apply`

```
# 等同于
def get_wencha(df):
    if df["bWendu"]-df["yWendu"]>10:
        return '温差大'
    else:
        return '温差正常'
df['wencha_type_2'] = df.apply(get_wencha,axis=1)

df['wencha_type_2'].value_counts()


"""输出如下"""
温差正常    187
温差大     178
Name: wencha_type_2, dtype: int64
```

6 五、Pandas数据统计函数

- 数据统计函数类型
    1. 汇总类统计
    2. 唯一去重和按值计数
    3. 相关系数和协方差

先读取数据：

```
fpath = "./datas/beijing_tianqi/beijing_tianqi_2018.csv"
df = pd.read_csv(fpath)
# 替换掉温度的后缀℃
df['bWendu'] = df['bWendu'].str.replace('℃','').astype('int32')
df['yWendu'] = df['yWendu'].str.replace('℃','').astype('int32')
df.head(3)
```

	ymd	bWendu	yWendu	tianqi	fengxiang	fengli	aqi	aqiInfo	aqiLevel
0	2018-01-01	3	-6	晴~多云	东北风	1-2级	59	良	2
1	2018-01-02	2	-5	阴~多云	东北风	1-2级	49	优	1
2	2018-01-03	2	-5	多云	北风	1-2级	28	优	1

6.1 汇总类统计

- 汇总类统计

```
df.describe().T

"""输出如下"""
```

	count	mean	std	min	25%	50%	75%	max
bWendu	365.0	18.665753	11.858046	-5.0	8.0	21.0	29.0	38.0
yWendu	365.0	8.358904	11.755053	-12.0	-3.0	8.0	19.0	27.0
aqi	365.0	82.183562	51.936159	21.0	46.0	69.0	104.0	387.0
aqiLevel	365.0	2.090411	1.029798	1.0	1.0	2.0	3.0	6.0

查看单个Series的数据(平均值)

```
## 查看单个Series的数据(平均值)
df["bWendu"].mean()

"""输出如下"""
18.66575342465753
```

最大值

```
# 最高温
df["bWendu"].max()

"""输出如下"""
38
```

最小值

```
# 最低温
df["bWendu"].min()

"""输出如下"""
-5
```

6.2 唯一去重和按值计数

6.2.1 唯一性去重

- 一般不用于数值列，而是枚举、分类列

风向

```
df["fengxiang"].unique()

"""输出如下"""
array(['东北风', '北风', '西北风', '西南风', '南风', '东南风', '东风', '西风'], dtype=object)
```

天气

```
df["tianqi"].unique()

"""输出如下"""
array(['晴~多云', '阴~多云', '多云', '阴', '多云~晴', '多云~阴', '晴', '阴~小雪', '小雪~多云',
       '小雨~阴', '小雨~雨夹雪', '多云~小雨', '小雨~多云', '大雨~小雨', '小雨', '阴~小雨',
       '多云~雷阵雨', '雷阵雨~多云', '阴~雷阵雨', '雷阵雨', '雷阵雨~大雨', '中雨~雷阵雨', '小雨~大雨',
       '暴雨~雷阵雨', '雷阵雨~中雨', '小雨~雷阵雨', '雷阵雨~阴', '中雨~小雨', '小雨~中雨', '雾~多云',
       '霾'], dtype=object)
```

风力

```
df["fengli"].unique()

"""输出如下"""
array(['1-2级', '4-5级', '3-4级', '2级', '1级', '3级'], dtype=object)
```

6.2.2 按值计数

- `value_counts()`

  风向

```
df["fengxiang"].value_counts()

"""输出如下"""
南风      92
西南风     64
北风      54
西北风     51
东南风     46
东北风     38
东风      14
西风       6
Name: fengxiang, dtype: int64
```

  天气

```
df["tianqi"].value_counts()

"""输出如下"""
晴          101
多云          95
多云~晴        40
晴~多云        34
多云~雷阵雨      14
多云~阴        10
雷阵雨           8
小雨~多云        8
阴~多云          8
雷阵雨~多云       7
小雨           6
多云~小雨        5
阴            4
雷阵雨~中雨       4
中雨~小雨        2
中雨~雷阵雨       2
阴~小雨         2
霾            2
雷阵雨~大雨       1
阴~雷阵雨        1
小雨~阴         1
```

```
雾~多云          1
小雪~多云         1
阴~小雪          1
小雨~中雨         1
小雨~雷阵雨        1
雷阵雨~阴         1
小雨~雨夹雪        1
大雨~小雨         1
暴雨~雷阵雨        1
小雨~大雨         1
Name: tianqi, dtype: int64
```

风力

```
df["fengli"].value_counts()

"""输出如下"""
1-2级     236
3-4级      68
1级        21
4-5级      20
2级        13
3级         7
Name: fengli, dtype: int64
```

6.3 相关系数和协方差

- 用途（超级厉害）：
    1. 两只股票，是不是同涨同跌？程度多大？正相关还是负相关？
    2. 产品销量的波动，跟哪些因素正相关、负相关，程度有多大？
- 来自知乎，对于两个变量X、Y：
    1. 协方差：***衡量同向反向程度***，如果协方差为正，说明X，Y同向变化，协方差越大说明同向程度越高；如果协方差为负，说明X，Y反向运动，协方差越小说明反向程度越高。
    2. 相关系数：***衡量相似度程度***，当他们的相关系数为1时，说明两个变量变化时的正向相似度最大，当相关系数为－1时，说明两个变量变化的反向相似度最大

6.3.1 协方差矩阵

- 协方差

```
# 协方差矩阵：
df.cov()

"""输出如下"""
```

	bWendu	yWendu	aqi	aqiLevel
bWendu	140.613247	135.529633	47.462622	0.879204
yWendu	135.529633	138.181274	16.186685	0.264165
aqi	47.462622	16.186685	2697.364564	50.749842
aqiLevel	0.879204	0.264165	50.749842	1.060485

6.3.2 相关系数矩阵

- 相关系数

```
# 相关系数矩阵

df.corr()

"""输出如下"""
```

	bWendu	yWendu	aqi	aqiLevel
bWendu	1.000000	0.972292	0.077067	0.071999
yWendu	0.972292	1.000000	0.026513	0.021822
aqi	0.077067	0.026513	1.000000	0.948883
aqiLevel	0.071999	0.021822	0.948883	1.000000

6.3.3 单独查看某两个特征的相关系数

- 单独查看某两个特征的相关系数

  单独查看空气质量和最高温度的相关系数

```
# 单独查看空气质量和最高温度的相关系数

df["aqi"].corr(df["bWendu"])

"""输出如下"""
0.07706705916811071
```

  单独查看空气质量和最低温度的相关系数

```
# 单独查看空气质量和最低温度的相关系数

df["aqi"].corr(df["yWendu"])

"""输出如下"""
0.026513282672968843
```

  单独查看空气质量和温差的相关系数

```python
# 空气质量和温差的相关系数

df["aqi"].corr(df["bWendu"]-df["yWendu"])

"""输出如下"""
0.2165225757638205
```

7 六、Pandas对缺失值的处理

- Pandas使用这些函数处理缺失值：
    1. `isnull` 和 `notnull`： 检测是否是空值，可用于 `df` 和 `series`
    2. `dropna`： 丢弃、删除缺失值
    3. `axis` : 删除行还是列，{0 or 'index', 1 or 'columns'}, default 0
    4. `how` : 如果等于any则任何值为空都删除，如果等于all则所有值都为空才删除 inplace : 如果为True则修改当前df，否则返回新的df
    5. `fillna`： 填充空值
        - `value`： 用于填充的值，可以是单个值，或者字典（key是列名，value是值）
        - `method` : 等于ffill使用前一个不为空的值填充forword fill；等于bfill使用后一个不为空的值填充 `backwordfill`
        - `axis` : 按行还是列填充，{0 or 'index', 1 or 'columns'}
        - `inplace` : 如果为True则修改当前df，否则返回新的df

7.1 实例：特殊Excel的读取、清洗、处理

7.1.1 步骤1：读取excel的时候，忽略前几个空行

- 读取excel的时候，忽略前几个空行

```
studf = pd.read_excel("./datas/student_excel/student_excel.xlsx")
studf.head()
```

	Unnamed: 0	Unnamed: 1	Unnamed: 2	Unnamed: 3
0	NaN	NaN	NaN	NaN
1	NaN	姓名	科目	分数
2	NaN	小明	语文	85
3	NaN	NaN	数学	80
4	NaN	NaN	英语	90
5	NaN	NaN	NaN	NaN
6	NaN	小王	语文	85
7	NaN	NaN	数学	NaN
8	NaN	NaN	英语	90
9	NaN	NaN	NaN	NaN
10	NaN	小刚	语文	85
11	NaN	NaN	数学	80
12	NaN	NaN	英语	90

忽略前面两个空行读入

```
# 忽略前面两个空行读入

studf = pd.read_excel("./datas/student_excel/student_excel.xlsx", skiprows=2)
studf.head()
```

	Unnamed: 0	姓名	科目	分数
0	NaN	小明	语文	85.0
1	NaN	NaN	数学	80.0
2	NaN	NaN	英语	90.0
3	NaN	NaN	NaN	NaN
4	NaN	小王	语文	85.0
5	NaN	NaN	数学	NaN
6	NaN	NaN	英语	90.0
7	NaN	NaN	NaN	NaN
8	NaN	小刚	语文	85.0
9	NaN	NaN	数学	80.0
10	NaN	NaN	英语	90.0

7.1.2 步骤2：检测空值

- 步骤2：检测空值

```
studf.isnull()

"""输出如下"""
```

	Unnamed: 0	姓名	科目	分数
0	True	False	False	False
1	True	True	False	False
2	True	True	False	False
3	True	True	True	True
4	True	False	False	False
5	True	True	False	True
6	True	True	False	False
7	True	True	True	True
8	True	False	False	False
9	True	True	False	False
10	True	True	False	False

分数为空的值：

```
studf["分数"].isnull()

"""输出如下"""
0      False
1      False
2      False
3       True
4      False
5       True
6      False
7       True
8      False
9      False
10     False
Name: 分数, dtype: bool
```

分数不为空的值：

```
studf["分数"].notnull()

"""输出如下"""
0       True
1       True
2       True
```

```
3      False
4       True
5      False
6       True
7      False
8       True
9       True
10      True
Name: 分数, dtype: bool
```

筛选没有空分数的所有行

```
# 筛选没有空分数的所有行

studf.loc[studf["分数"].notnull(), :]

"""输出如下"""
```

	Unnamed: 0	姓名	科目	分数
0	NaN	小明	语文	85.0
1	NaN	NaN	数学	80.0
2	NaN	NaN	英语	90.0
4	NaN	小王	语文	85.0
6	NaN	NaN	英语	90.0
8	NaN	小刚	语文	85.0
9	NaN	NaN	数学	80.0
10	NaN	NaN	英语	90.0

7.1.3 步骤3：删除掉全是空值的列

- 删除掉全是空值的列

```
studf.dropna(axis="columns", how='all', inplace=True)
# 或者
studf.dropna(axis=1, how='all', inplace=True)
studf

"""输出如下"""
```

	姓名	科目	分数
0	小明	语文	85.0
1	NaN	数学	80.0
2	NaN	英语	90.0
4	小王	语文	85.0
5	NaN	数学	NaN
6	NaN	英语	90.0
8	小刚	语文	85.0
9	NaN	数学	80.0
10	NaN	英语	90.0

7.1.4 步骤4：删除掉全是空值的行

- 删除掉全是空值的行

```
studf.dropna(axis=0,how='all',inplace=True)
studf

"""输出如下"""
```

	姓名	科目	分数
0	小明	语文	85.0
1	NaN	数学	80.0
2	NaN	英语	90.0
4	小王	语文	85.0
5	NaN	数学	NaN
6	NaN	英语	90.0
8	小刚	语文	85.0
9	NaN	数学	80.0
10	NaN	英语	90.0

7.1.5 步骤5：将分数列为空的填充为0分

- 将分数列为空的填充为0分

```python
studf['分数'] = studf['分数'].fillna(0)

# 等同于

studf.fillna({"分数":0})

# 等同于

studf.loc[:, '分数'] = studf['分数'].fillna(0)

studf
"""输出如下"""
```

	姓名	科目	分数
0	小明	语文	85.0
1	NaN	数学	80.0
2	NaN	英语	90.0
4	小王	语文	85.0
5	NaN	数学	0.0
6	NaN	英语	90.0
8	小刚	语文	85.0
9	NaN	数学	80.0
10	NaN	英语	90.0

7.1.6 步骤6：将姓名的缺失值填充

- 使用前面的有效值填充，用 `ffill`： forward fill

```python
studf.loc[:, '姓名'] = studf['姓名'].fillna(method="ffill")
studf

"""输出如下"""
```

	姓名	科目	分数
0	小明	语文	85.0
1	小明	数学	80.0
2	小明	英语	90.0
4	小王	语文	85.0
5	小王	数学	0.0
6	小王	英语	90.0
8	小刚	语文	85.0
9	小刚	数学	80.0
10	小刚	英语	90.0

8 七、Pandas的SettingWithCopyWarning报警

8.1 读取数据

- 读取数据

```
import pandas as pd

fpath = "./datas/beijing_tianqi/beijing_tianqi_2018.csv"
df = pd.read_csv(fpath)

# 替换掉温度的后缀℃
df.loc[:, "bWendu"] = df["bWendu"].str.replace("℃", "").astype('int32')
df.loc[:, "yWendu"] = df["yWendu"].str.replace("℃", "").astype('int32')

df.head()
```

	ymd	bWendu	yWendu	tianqi	fengxiang	fengli	aqi	aqiInfo	aqiLevel
0	2018-01-01	3	-6	晴~多云	东北风	1-2级	59	良	2
1	2018-01-02	2	-5	阴~多云	东北风	1-2级	49	优	1
2	2018-01-03	2	-5	多云	北风	1-2级	28	优	1
3	2018-01-04	0	-8	阴	东北风	1-2级	28	优	1
4	2018-01-05	3	-6	多云~晴	西北风	1-2级	50	优	1

8.2 复现

- 复现 `SettingWithCopyWarning`

```
# 只选出3月份的数据用于分析

condition = df["ymd"].str.startswith("2018-03")
```

```
# 设置温差d
f[condition]["wen_cha"] = df["bWendu"]-df["yWendu"]
```

```
"""警告内容"""

d:\appdata\python37\lib\site-packages\ipykernel_launcher.py:2:
SettingWithCopyWarning: A value is trying to be set on a copy of a
slice from a DataFrame.Try using .loc[row_indexer,col_indexer] =
value insteadSee the caveats in the documentation:
http://pandas.pydata.org/pandas-
docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
```

```
# 查看是否修改成功
df[condition].head()
```

	ymd	bWendu	yWendu	tianqi	fengxiang	fengli	aqi	aqiInfo	aqiLevel
59	2018-03-01	8	-3	多云	西南风	1-2级	46	优	1
60	2018-03-02	9	-1	晴~多云	北风	1-2级	95	良	2
61	2018-03-03	13	3	多云~阴	北风	1-2级	214	重度污染	5
62	2018-03-04	7	-2	阴~多云	东南风	1-2级	144	轻度污染	3
63	2018-03-05	8	-3	晴	南风	1-2级	94	良	2

8.3 原因

- 发出警告的代码
  `df[condition]["wen_cha"] = df["bWendu"]-df["yWendu"]`

  相当于：`df.get(condition).set(wen_cha)`，第一步骤的get发出了报警

  链式操作其实是两个步骤，先get后set，get得到的dataframe可能是view也可能是copy，pandas发出警告

  官网文档：

  > https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

  核心要诀：`pandas` 的 `dataframe` 的修改写操作，只允许在源 `dataframe` 上进行，一步到位

8.4 解决方法1

- 将 `get+set` 的两步操作，改成 `set` 的一步操作

```
df.loc[condition, "wen_cha"] = df["bWendu"]-df["yWendu"]
df[condition].head()
```

	ymd	bWendu	yWendu	tianqi	fengxiang	fengli	aqi	aqiInfo	aqiLevel	wen_cha
59	2018-03-01	8	-3	多云	西南风	1-2级	46	优	1	11.0
60	2018-03-02	9	-1	晴~多云	北风	1-2级	95	良	2	10.0
61	2018-03-03	13	3	多云~阴	北风	1-2级	214	重度污染	5	10.0
62	2018-03-04	7	-2	阴~多云	东南风	1-2级	144	轻度污染	3	9.0
63	2018-03-05	8	-3	晴	南风	1-2级	94	良	2	11.0

8.5 解决方法2

- 如果需要预筛选数据做后续的处理分析，使用 `copy` 复制 `dataframe`

```
df_month3 = df[condition].copy()
df_month3.head()
```

	ymd	bWendu	yWendu	tianqi	fengxiang	fengli	aqi	aqiInfo	aqiLevel	wen_cha
59	2018-03-01	8	-3	多云	西南风	1-2级	46	优	1	11.0
60	2018-03-02	9	-1	晴~多云	北风	1-2级	95	良	2	10.0
61	2018-03-03	13	3	多云~阴	北风	1-2级	214	重度污染	5	10.0
62	2018-03-04	7	-2	阴~多云	东南风	1-2级	144	轻度污染	3	9.0
63	2018-03-05	8	-3	晴	南风	1-2级	94	良	2	11.0

```
df_month3["wen_cha"] = df["bWendu"]-df["yWendu"]
df_month3.head()
```

	ymd	bWendu	yWendu	tianqi	fengxiang	fengli	aqi	aqiInfo	aqiLevel	wen_cha
59	2018-03-01	8	-3	多云	西南风	1-2级	46	优	1	11
60	2018-03-02	9	-1	晴~多云	北风	1-2级	95	良	2	10
61	2018-03-03	13	3	多云~阴	北风	1-2级	214	重度污染	5	10
62	2018-03-04	7	-2	阴~多云	东南风	1-2级	144	轻度污染	3	9
63	2018-03-05	8	-3	晴	南风	1-2级	94	良	2	11

- 总之， `pandas` 不允许先筛选子 `dataframe`， 再进行修改写入, 要么使用 `.loc` 实现一个步骤直接修改源 `dataframe` ,要么先复制一个子 `dataframe` 再一个步骤执行修改

9 八、pandas数据排序

- Series的排序：

  `Series.sort_values(ascending=True, inplace=False)`
  参数说明：

  1. `ascending`：默认为True升序排序，为False降序排序
  2. `inplace`：是否修改原始Series

- DataFrame的排序：

  `DataFrame.sort_values(by, ascending=True, inplace=False)`
  参数说明：

  1. `by`：字符串或者List<字符串>，单列排序或者多列排序
  2. `ascending`：bool或者List，升序还是降序，如果是list对应by的多列
  3. `inplace`：是否修改原始DataFrame

```
fpath = "./datas/beijing_tianqi/beijing_tianqi_2018.csv"
df = pd.read_csv(fpath)

# 替换掉温度的后缀℃
df.loc[:, "bWendu"] = df["bWendu"].str.replace("℃", "").astype('int32')
df.loc[:, "yWendu"] = df["yWendu"].str.replace("℃", "").astype('int32')

df.head()
```

	ymd	bWendu	yWendu	tianqi	fengxiang	fengli	aqi	aqiInfo	aqiLevel
0	2018-01-01	3	-6	晴~多云	东北风	1-2级	59	良	2
1	2018-01-02	2	-5	阴~多云	东北风	1-2级	49	优	1
2	2018-01-03	2	-5	多云	北风	1-2级	28	优	1
3	2018-01-04	0	-8	阴	东北风	1-2级	28	优	1
4	2018-01-05	3	-6	多云~晴	西北风	1-2级	50	优	1

9.1 Series的排序

- `Series` 的排序

  按 `aqi` 升序排序

```python
df['aqi'].sort_values()

"""输出入下"""
```

```
271     21
281     21
249     22
272     22
301     22
       ...
317    266
71     287
91     287
72     293
86     387
Name: aqi, Length: 365, dtype: int64
```

按 `aqi` 降序排序

```python
df['aqi'].sort_values(ascending=False)

"""输出如下"""
```

```
86     387
72     293
91     287
71     287
317    266
       ...
301     22
272     22
249     22
281     21
271     21
Name: aqi, Length: 365, dtype: int64
```

按 `天气` 升序排序

```python
df["tianqi"].sort_values()

"""输出如下"""
```

```
225      中雨~小雨
230      中雨~小雨
197    中雨~雷阵雨
196    中雨~雷阵雨
112         多云
          ...
191    雷阵雨~大雨
219     雷阵雨~阴
335      雾~多云
353          霾
348          霾
Name: tianqi, Length: 365, dtype: object
```

9.2 DataFrame的排序

9.2.1 单列排序

- 单列排序

  按 `aqi` 升序排序

```
df.sort_values(by='aqi')

"""输出如下"""
```

	ymd	bWendu	yWendu	tianqi	fengxiang	fengli	aqi	aqiInfo	aqiLevel
271	2018-09-29	22	11	晴	北风	3-4级	21	优	1
281	2018-10-09	15	4	多云~晴	西北风	4-5级	21	优	1
249	2018-09-07	27	16	晴	西北风	3-4级	22	优	1
272	2018-09-30	19	13	多云	西北风	4-5级	22	优	1
301	2018-10-29	15	3	晴	北风	3-4级	22	优	1
...	...	...	...	...	...	...	...	...	...
317	2018-11-14	13	5	多云	南风	1-2级	266	重度污染	5
71	2018-03-13	17	5	晴~多云	南风	1-2级	287	重度污染	5
91	2018-04-02	26	11	多云	北风	1-2级	287	重度污染	5
72	2018-03-14	15	6	多云~阴	东北风	1-2级	293	重度污染	5
86	2018-03-28	25	9	多云~晴	东风	1-2级	387	严重污染	6

365 rows × 9 columns

  按 `aqi` 降序排序

```python
df.sort_values(by="aqi", ascending=False)

"""输出如下"""
```

	ymd	bWendu	yWendu	tianqi	fengxiang	fengli	aqi	aqiInfo	aqiLevel
86	2018-03-28	25	9	多云~晴	东风	1-2级	387	严重污染	6
72	2018-03-14	15	6	多云~阴	东北风	1-2级	293	重度污染	5
71	2018-03-13	17	5	晴~多云	南风	1-2级	287	重度污染	5
91	2018-04-02	26	11	多云	北风	1-2级	287	重度污染	5
317	2018-11-14	13	5	多云	南风	1-2级	266	重度污染	5
...	...	...	...	...	...	...	...	...	...
249	2018-09-07	27	16	晴	西北风	3-4级	22	优	1
301	2018-10-29	15	3	晴	北风	3-4级	22	优	1
272	2018-09-30	19	13	多云	西北风	4-5级	22	优	1
271	2018-09-29	22	11	晴	北风	3-4级	21	优	1
281	2018-10-09	15	4	多云~晴	西北风	4-5级	21	优	1

365 rows × 9 columns

9.2.2 多列排序

- 多列排序

  按空气质量等级、最高温度排序，默认升序

```python
# 按空气质量等级、最高温度排序，默认升序

df.sort_values(by=['aqiLevel','bWendu'])

"""输出如下"""
```

	ymd	bWendu	yWendu	tianqi	fengxiang	fengli	aqi	aqiInfo	aqiLevel
360	2018-12-27	-5	-12	多云~晴	西北风	3级	48	优	1
22	2018-01-23	-4	-12	晴	西北风	3-4级	31	优	1
23	2018-01-24	-4	-11	晴	西南风	1-2级	34	优	1
340	2018-12-07	-4	-10	晴	西北风	3级	33	优	1
21	2018-01-22	-3	-10	小雪~多云	东风	1-2级	47	优	1
...	...	...	...	...	...	...	...	...	...
71	2018-03-13	17	5	晴~多云	南风	1-2级	287	重度污染	5
90	2018-04-01	25	11	晴~多云	南风	1-2级	218	重度污染	5
91	2018-04-02	26	11	多云	北风	1-2级	287	重度污染	5
85	2018-03-27	27	11	晴	南风	1-2级	243	重度污染	5
86	2018-03-28	25	9	多云~晴	东风	1-2级	387	严重污染	6

365 rows × 9 columns

按空气质量等级、最高温度排序,两个字段都是降序

```
# 按空气质量等级、最高温度排序,两个字段都是降序
df.sort_values(by=['aqiLevel','bWendu'],ascending=False)

"""输出如下"""
```

	ymd	bWendu	yWendu	tianqi	fengxiang	fengli	aqi	aqiInfo	aqiLevel
86	2018-03-28	25	9	多云~晴	东风	1-2级	387	严重污染	6
85	2018-03-27	27	11	晴	南风	1-2级	243	重度污染	5
91	2018-04-02	26	11	多云	北风	1-2级	287	重度污染	5
90	2018-04-01	25	11	晴~多云	南风	1-2级	218	重度污染	5
71	2018-03-13	17	5	晴~多云	南风	1-2级	287	重度污染	5
...	...	...	...	...	...	...	...	...	...
362	2018-12-29	-3	-12	晴	西北风	2级	29	优	1
22	2018-01-23	-4	-12	晴	西北风	3-4级	31	优	1
23	2018-01-24	-4	-11	晴	西南风	1-2级	34	优	1
340	2018-12-07	-4	-10	晴	西北风	3级	33	优	1
360	2018-12-27	-5	-12	多云~晴	西北风	3级	48	优	1

365 rows × 9 columns

分别指定升序和降序 ,前面升序，后面降序

```
# 分别指定升序和降序 ,前面升序，后面降序
df.sort_values(by=['aqiLevel','bWendu'], ascending=[True,False])
"""输出如下"""
```

	ymd	bWendu	yWendu	tianqi	fengxiang	fengli	aqi	aqiInfo	aqiLevel
178	2018-06-28	35	24	多云~晴	北风	1-2级	33	优	1
149	2018-05-30	33	18	晴	西风	1-2级	46	优	1
206	2018-07-26	33	25	多云~雷阵雨	东北风	1-2级	40	优	1
158	2018-06-08	32	19	多云~雷阵雨	西南风	1-2级	43	优	1
205	2018-07-25	32	25	多云	北风	1-2级	28	优	1
...	...	...	...	...	...	...	...	...	...
317	2018-11-14	13	5	多云	南风	1-2级	266	重度污染	5
329	2018-11-26	10	0	多云	东南风	1级	245	重度污染	5
335	2018-12-02	9	2	雾~多云	东北风	1级	234	重度污染	5
57	2018-02-27	7	0	阴	东风	1-2级	220	重度污染	5
86	2018-03-28	25	9	多云~晴	东风	1-2级	387	严重污染	6

365 rows × 9 columns

10 九、Pandas字符串处理

- 前面我们已经使用了字符串的处理函数：

  `df["bWendu"].str.replace("℃", "").astype('int32')`

  Pandas的字符串处理：

  1. 使用方法：先获取Series的str属性，然后在属性上调用函数；
  2. 只能在字符串列上使用，不能数字列上使用；
  3. Dataframe上没有str属性和处理方法
  4. Series.str并不是Python原生字符串，而是自己的一套方法，不过大部分和原生str很相似；

- Series.str字符串方法列表参考文档:

  > https://pandas.pydata.org/pandas-docs/stable/reference/series.html#string-handling

- 本节演示内容：

  1. 获取Series的str属性，然后使用各种字符串处理函数（注意：str是Series上的属性）
  2. 使用str的startswith、contains等bool类Series可以做条件查询
  3. 需要多次str处理的链式操作
  4. 使用正则表达式的处理

```
fpath = "./datas/beijing_tianqi/beijing_tianqi_2018.csv"
df = pd.read_csv(fpath)
df.head()

"""输出如下"""
```

	ymd	bWendu	yWendu	tianqi	fengxiang	fengli	aqi	aqiInfo	aqiLevel
0	2018-01-01	3°C	-6°C	晴~多云	东北风	1-2级	59	良	2
1	2018-01-02	2°C	-5°C	阴~多云	东北风	1-2级	49	优	1
2	2018-01-03	2°C	-5°C	多云	北风	1-2级	28	优	1
3	2018-01-04	0°C	-8°C	阴	东北风	1-2级	28	优	1
4	2018-01-05	3°C	-6°C	多云~晴	西北风	1-2级	50	优	1

```
df.dtypes

"""输出如下"""
ymd           object
bWendu        object
yWendu        object
tianqi        object
fengxiang     object
fengli        object
aqi            int64
aqiInfo       object
aqiLevel       int64
dtype: object
```

10.1 Series的str属性

- 获取Series的str属性，使用各种字符串处理函数

```
df["bWendu"].str

"""输出如下"""
<pandas.core.strings.StringMethods at 0x110856be0>
```

字符串替换函数

```
# 字符串替换函数
df["bWendu"].str.replace("℃", "")

"""输出如下"""
0        3
1        2
2        2
3        0
4        3
        ..
360     -5
361     -3
362     -3
363     -2
364     -2
Name: bWendu, Length: 365, dtype: object
```

判断是不是数字

```
# 判断是不是数字
# df["bWendu"].str.isnumeric()
df["bWendu"].str.isdigit()

"""输出如下"""

```

属性只能在是字符串的Series上使用，否则会报错

```
# fengxiang的dtype是object
df['fengxiang'].str.len()

"""输出如下"""
0       3
1       3
2       2
3       3
4       3
       ..
360     3
361     3
362     3
363     3
364     3
Name: fengxiang, Length: 365, dtype: int64
```

```
# aqi的dtype是int64
df["aqi"].str.len()

"""输出如下"""
AttributeError: Can only use .str accessor with string values!
```

10.2 字符串条件查询

- 使用str的startswith、contains等得到bool的Series可以做条件查询

```
condition = df['ymd'].str.startswith('2018-03')
condition

"""输出如下"""
0       False
1       False
2       False
3       False
4       False
        ...
360     False
361     False
```

```
362    False
363    False
364    False
Name: ymd, Length: 365, dtype: bool
```

```
df[condition].head()

"""输出如下"""
```

	ymd	bWendu	yWendu	tianqi	fengxiang	fengli	aqi	aqiInfo	aqiLevel
59	2018-03-01	8°C	-3°C	多云	西南风	1-2级	46	优	1
60	2018-03-02	9°C	-1°C	晴~多云	北风	1-2级	95	良	2
61	2018-03-03	13°C	3°C	多云~阴	北风	1-2级	214	重度污染	5
62	2018-03-04	7°C	-2°C	阴~多云	东南风	1-2级	144	轻度污染	3
63	2018-03-05	8°C	-3°C	晴	南风	1-2级	94	良	2

10.3 需要多次str处理的链式操作

- 怎样提取201803这样的数字月份?
    1. 先将日期2018-03-31替换成20180331的形式
    2. 提取月份字符串201803
- 先将日期2018-03-31替换成20180331的形式

```
df['ymd'].str.replace('-','')

"""输出如下"""
0       20180101
1       20180102
2       20180103
3       20180104
4       20180105
           ...
360     20181227
361     20181228
362     20181229
363     20181230
364     20181231
Name: ymd, Length: 365, dtype: object
```

- 提取月份字符串201803

  注意:

```
# 每次调用函数，都返回一个新Series,str是Series上的属性，所有每次都需要重新调用str属性
# 否则会报错
df["ymd"].str.replace("-", "").slice(0, 6)

"""输出如下"""
AttributeError: 'Series' object has no attribute 'slice'
```

重新调用str属性

```
# 重新调用str属性
df["ymd"].str.replace("-", "").str.slice(0, 6)
# 等价于
# df["ymd"].str.replace("-", "").str[0:6]

"""输出如下"""
0      201801
1      201801
2      201801
3      201801
4      201801
         ...
360    201812
361    201812
362    201812
363    201812
364    201812
Name: ymd, Length: 365, dtype: object
```

slice就是切片语法，可以直接用

```
# slice就是切片语法，可以直接用
df["ymd"].str.replace("-", "").str[0:6]

"""输出如下"""
0      201801
1      201801
2      201801
3      201801
4      201801
         ...
360    201812
361    201812
362    201812
363    201812
364    201812
Name: ymd, Length: 365, dtype: object
```

10.4 正则表达式

- 添加新列

```
# 添加新列
def get_nianyueri(x):
    """获取年月日"""
    year,month,day = x["ymd"].split("-")
    return "{}年{}月{}日".format(year,month,day)
    # 或者
    # return f"{year}年{month}月{day}日"
df['中文日期'] = df.apply(get_nianyueri,axis=1)
df.head()

"""输出如下"""
```

	ymd	bWendu	yWendu	tianqi	fengxiang	fengli	aqi	aqiInfo	aqiLevel	中文日期
0	2018-01-01	3°C	-6°C	晴~多云	东北风	1-2级	59	良	2	2018年01月01日
1	2018-01-02	2°C	-5°C	阴~多云	东北风	1-2级	49	优	1	2018年01月02日
2	2018-01-03	2°C	-5°C	多云	北风	1-2级	28	优	1	2018年01月03日
3	2018-01-04	0°C	-8°C	阴	东北风	1-2级	28	优	1	2018年01月04日
4	2018-01-05	3°C	-6°C	多云~晴	西北风	1-2级	50	优	1	2018年01月05日

- 问题：怎样将“2018年12月31日”中的年、月、日三个中文字符去除?

  方法一：链式replace

```
# 方法一：链式replace
df['中文日期'].str.replace('年','').str.replace('月','').str.replace('日','')

"""输出如下"""
0       20180101
1       20180102
2       20180103
3       20180104
4       20180105
          ...
360     20181227
361     20181228
362     20181229
363     20181230
364     20181231
Name: 中文日期, Length: 365, dtype: object
```

  方法2：正则表达式替换

  Series.str默认就开启了正则表达式模式

```
# 方法2：正则表达式替换

df['中文日期'].str.replace('[年月日]','')


"""输出如下"""
0      20180101
1      20180102
2      20180103
3      20180104
4      20180105
         ...
360    20181227
361    20181228
362    20181229
363    20181230
364    20181231
Name: 中文日期, Length: 365, dtype: object
```

11 十、Pandas的axis参数怎么理解?

- axis=0或者"index"：
    - 如果是单行操作，就指的是某一行
    - 如果是聚合操作，指的是跨行cross rows
- axis=1或者"columns"：
    - 如果是单列操作，就指的是某一列
    - 如果是聚合操作，指的是跨列cross columns
- 按哪个axis，就是这个axis要动起来(类似被for遍历)，其它的axis保持不动

```
df = pd.DataFrame(
    np.arange(12).reshape(3,4),
    columns=['A', 'B', 'C', 'D']
)

df
"""输出如下"""
```

	A	B	C	D
0	0	1	2	3
1	4	5	6	7
2	8	9	10	11

11.1 单列drop

- 单列drop，就是删除某一列

```
# 代表的就是删除某列
df.drop('A',axis=1)

"""输出如下"""
```

	B	C	D
0	1	2	3
1	5	6	7
2	9	10	11

11.2 单行drop

- 单行drop，就是删除某一行

```
df
"""输出如下"""
```

	A	B	C	D
0	0	1	2	3
1	4	5	6	7
2	8	9	10	11

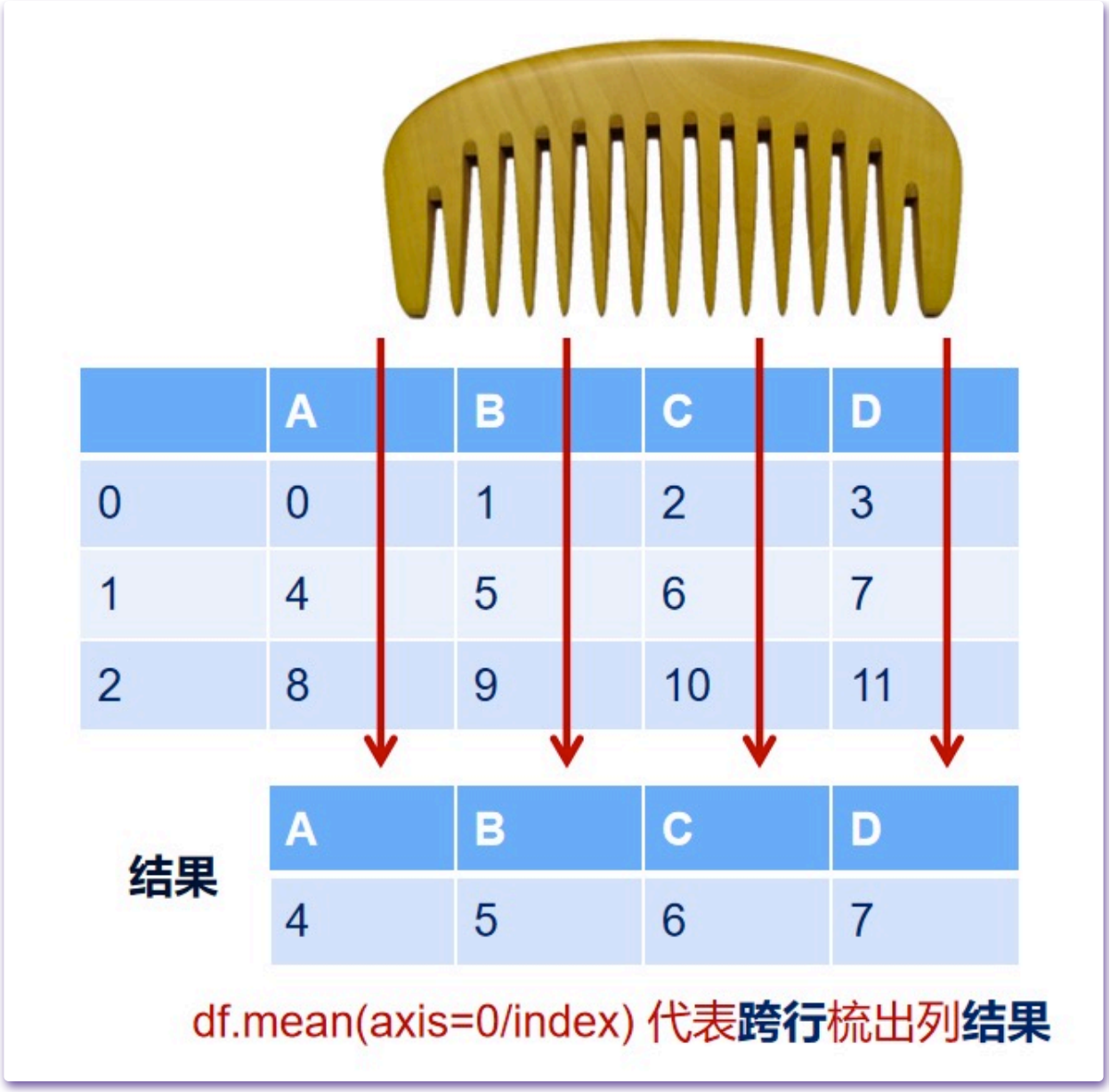
```
# 代表的就是删除某行
df.drop(1,axis=0)

"""输出如下"""
```

	A	B	C	D
0	0	1	2	3
2	8	9	10	11

11.3 按axis=0、index执行mean聚合操作

- 反直觉：输出的不是每行的结果，而是每列的结果




```
df

"""输出如下"""
```

	A	B	C	D
0	0	1	2	3
1	4	5	6	7
2	8	9	10	11

```python
# axis=0 or axis=index
df.mean(axis=0)

"""输出如下"""
A    4.0
B    5.0
C    6.0
D    7.0
dtype: float64
```

- 指定了按哪个axis，就是这个axis要动起来(类似被for遍历)，其它的axis保持不动

11.4 按axis=1、columns执行mean聚合操作

- 反直觉：输出的不是每行的结果，而是每列的结果

```python
df
```

	A	B	C	D
0	0	1	2	3
1	4	5	6	7
2	8	9	10	11

```python
# axis=1 or axis=columns
df.mean(axis=1)

"""输出如下"""
0    1.5
1    5.5
2    9.5
dtype: float64
```

- 指定了按哪个axis，就是这个axis要动起来(类似被for遍历)，其它的axis保持不动

11.5 再次举例，加深理解

- 求和：`apply` 方法

```python
def get_sum_value(x):
    return x["A"] + x["B"] + x["C"] + x["D"]

df["sum_value"] = df.apply(get_sum_value, axis=1)
df

"""输出如下"""
```

	A	B	C	D	sum_value
0	0	1	2	3	6
1	4	5	6	7	22
2	8	9	10	11	38

- 求和：`df.sum()`

```python
df['sum_value_1'] = df.sum(axis=1)

"""输出如下"""
df
```

	A	B	C	D	sum_value	sum_value_1
0	0	1	2	3	6	12
1	4	5	6	7	22	44
2	8	9	10	11	38	76

12 十一、Pandas索引作用和层次化索引

12.1 索引作用

- 把数据存储于普通的column列也能用于数据查询，那使用index有什么好处？

  index的用途总结：

  1. 更方便的数据查询；
  2. 使用index可以获得性能提升；
  3. 自动的数据对齐功能；
  4. 更多更强大的数据结构支持；

```
df = pd.read_csv("./datas/ml-latest-small/ratings.csv")
df.head()

"""输出如下"""
```

	userId	movieId	rating	timestamp
0	1	1	4.0	964982703
1	1	3	4.0	964981247
2	1	6	4.0	964982224
3	1	47	5.0	964983815
4	1	50	5.0	964982931

```
df.count()
"""输出如下"""

userId       100836
movieId      100836
rating       100836
timestamp    100836
dtype: int64
```

12.1.1 查询数据

- 使用index查询数据

```
df.head()

"""输出如下"""
```

	userId	movieId	rating	timestamp
0	1	1	4.0	964982703
1	1	3	4.0	964981247
2	1	6	4.0	964982224
3	1	47	5.0	964983815
4	1	50	5.0	964982931

```
# drop==False, 让索引列还保持在column
df.set_index("userId", inplace=True, drop=False)
df.head()

"""输出如下"""
```

	userId	movieId	rating	timestamp
userId				
1	1	1	4.0	964982703
1	1	3	4.0	964981247
1	1	6	4.0	964982224
1	1	47	5.0	964983815
1	1	50	5.0	964982931

```
df.index

"""输出如下"""
Int64Index([  1,   1,   1,   1,   1,   1,   1,   1,   1,   1,
             ...
             610, 610, 610, 610, 610, 610, 610, 610, 610, 610],
            dtype='int64', name='userId', length=100836)
```

```
# 使用index的查询方法
df.loc[500].head(5)

"""输出如下"""
```

	userId	movieId	rating	timestamp
userId				
500	500	1	4.0	1005527755
500	500	11	1.0	1005528017
500	500	39	1.0	1005527926
500	500	101	1.0	1005527980
500	500	104	4.0	1005528065

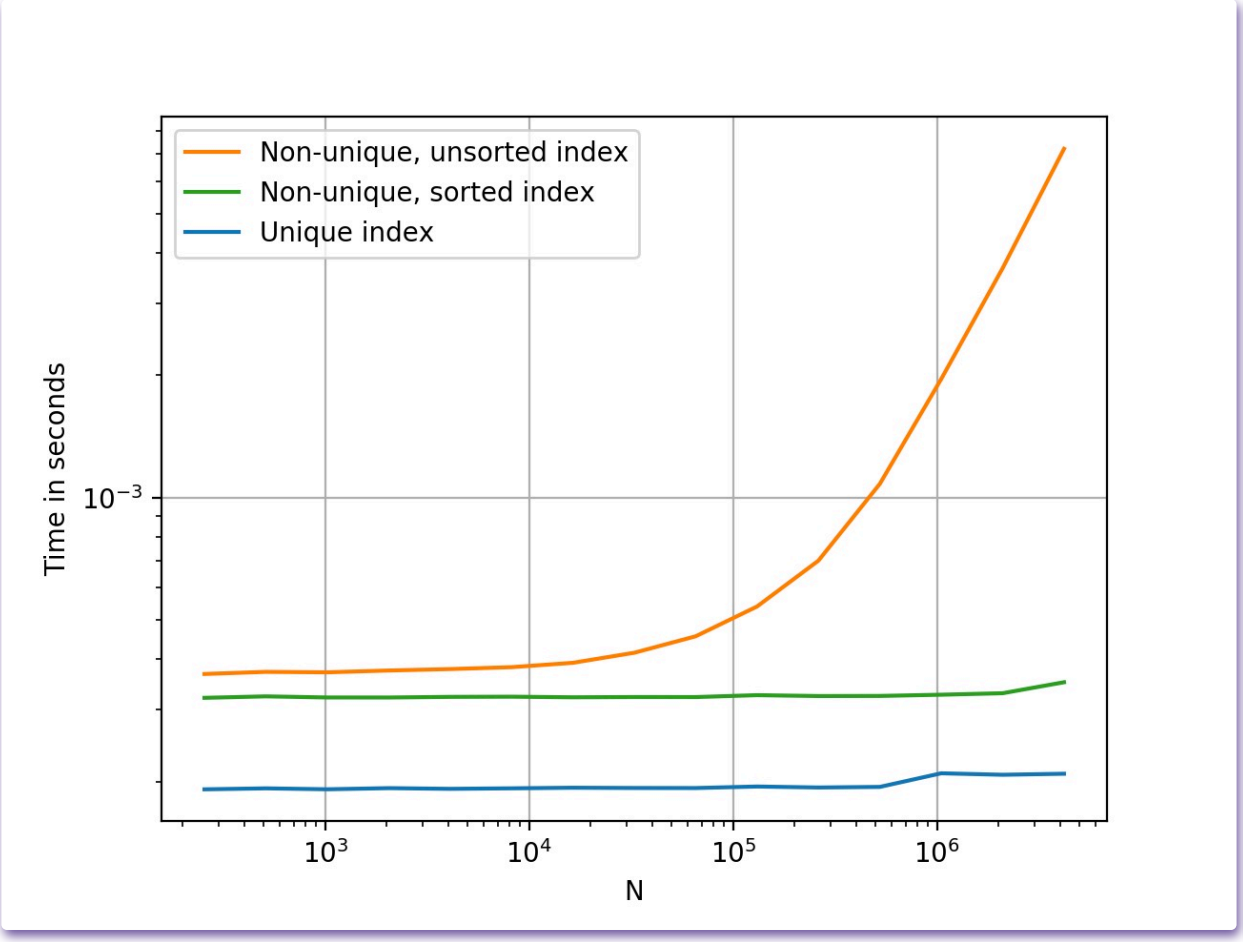
```python
# 使用column的condition查询方法
df.loc[df["userId"] == 500].head()

"""输出如下"""
```

	userId	movieId	rating	timestamp
userId				
500	500	1	4.0	1005527755
500	500	11	1.0	1005528017
500	500	39	1.0	1005527926
500	500	101	1.0	1005527980
500	500	104	4.0	1005528065

12.1.2 提升查询性能

- 使用index会提升查询性能
- 如果index是唯一的，Pandas会使用哈希表优化，查询性能为O(1);
- 如果index不是唯一的，但是有序，Pandas会使用二分查找算法，查询性能为O(logN);
- 如果index是完全随机的，那么每次查询都要扫描全表，查询性能为O(N);



12.1.3 实验1：完全随机的顺序查询

- 完全随机的顺序查询

  将数据随机打散

```
# 将数据随机打散

from sklearn.utils import shuffle
df_shuffle = shuffle(df)
df_shuffle.head()
"""输出如下"""
```

	userId	movieId	rating	timestamp
userId				
217	217	2366	3.0	955935853
200	200	46976	4.5	1229887105
542	542	47	5.0	1163386913
382	382	50	5.0	1515160611
84	84	899	4.0	858773054

  索引是否是递增的(单调递增)

```
# 索引是否是递增的(单调递增)

df_shuffle.index.is_monotonic_increasing

"""输出如下"""
False
```

  查看行索引值是否是唯一的

```
# 查看行索引值是否是唯一的（注意这一的航索引已经重新设置成userid，并且已经打乱了）
df_shuffle.index.is_unique

"""输出如下"""
False
```

  计时

```
# 计时，查询id==500数据性能
%timeit df_shuffle.loc[500]

"""输出如下"""
435 µs ± 11.5 µs per loop (mean ± std. dev. of 7 runs, 1000 loops each)
```

12.1.4 实验2：将index排序后的查询

- 将index排序后的查询

```
df_sorted = df_shuffle.sort_index()
df_sorted.head()

"""输出如下"""
```

	userId	movieId	rating	timestamp
userId				
1	1	1920	4.0	964981780
1	1	1377	3.0	964982653
1	1	2616	4.0	964983080
1	1	1278	5.0	964983414
1	1	3439	4.0	964981799

```
# 索引是否是递增的

df_sorted.index.is_monotonic_increasing

"""输出如下"""
True
```

```
df_sorted.index.is_unique

"""输出如下"""
False
```

```
%timeit df_sorted.loc[500]

"""输出如下"""
118 µs ± 15.8 µs per loop (mean ± std. dev. of 7 runs, 10000 loops each)
```

- 无序的是 435μs 有序的是 118μs.

12.1.5 自动对齐数据

- 使用 `index` 能自动对齐数据,包括 `series` 和 `dataframe`

```
s1 = pd.Series([1,2,3], index=list("abc"))
s1

"""输出如下"""
a    1
b    2
c    3
dtype: int64
```

```
s2 = pd.Series([2,3,4], index=list("bcd"))
s2

"""输出如下"""
b    2
c    3
d    4
dtype: int64
```

```
s1+s2

"""输出如下"""
a    NaN
b    4.0
c    6.0
d    NaN
dtype: float64
```

12.1.6 更多更强大的数据结构支持

- 很多强大的索引数据结构
    1. `CategoricalIndex`，基于分类数据的`Index`，提升性能；
    2. `MultiIndex`，多维索引，用于`groupby`多维聚合后结果等；
    3. `DatetimeIndex`，时间类型索引，强大的日期和时间的方法支持。

12.2 层次化索引

- 层次化索引（`hierarchical indexing`）是pandas的一项重要功 能，它使你能在一个轴上拥有多个（两个以上）索引级别。抽象 点说，它使你能以低维度形式处理高维度数据。

12.2.1 层次化索引（hierarchical indexing）

- 创建层次化索引(`series`)

```
# 创建层次化索引(series)
data = pd.Series(np.random.randn(9),
                 index=[['a', 'a', 'a', 'b', 'b', 'c', 'c', 'd', 'd'],
                        [1, 2, 3, 1, 3, 1, 2, 2, 3]])
data

"""输出如下"""
a  1   -0.204708
   2    0.478943
   3   -0.519439
```

```
b  1   -0.555730
   3    1.965781
c  1    1.393406
   2    0.092908
d  2    0.281746
   3    0.769023
dtype: float64
```

查看 `MultiIndex`

```
# 查看MultiIndex
data.index

"""输出如下"""
MultiIndex(levels=[['a', 'b', 'c', 'd'], [1, 2, 3]],
           labels=[[0, 0, 0, 1, 1, 2, 2, 3, 3], [0, 1, 2, 0, 2, 0, 1, 1, 2]])
```

- 对于一个层次化索引对象，可以使用所谓 `部分索引`，使用它选取数据子集的操作更简单

```
data['b']

"""输出如下"""
1   -0.555730
3    1.965781
dtype: float64
```

```
data['b':'c']

"""输出如下"""
b  1   -0.555730
   3    1.965781
c  1    1.393406
   2    0.092908
dtype: float64
```

```
data.loc[['b', 'd']]

"""输出如下"""
b  1   -0.555730
   3    1.965781
d  2    0.281746
   3    0.769023
dtype: float64
```

可以在内层选取（稍微难理解一点）

```
# 可以在内层选取（稍微难理解一点）
data.loc[:, 2]

"""输出如下"""
a    0.478943
c    0.092908
d    0.281746
dtype: float64
```

层次化索引在数据重塑和基于分组的操作（如透视表生成）中扮演着重要的角色

```
# 层次化索引在数据重塑和基于分组的操作（如透视表生成）中扮演着重要的角色。例如，可以通过unstack方法将这段数据重新安排到一个DataFrame中：
data.unstack()
```

	1	2	3
a	-0.204708	0.478943	-0.519439
b	-0.555730	NaN	1.965781
c	1.393406	0.092908	NaN
d	NaN	0.281746	0.769023

`unstack` 的逆运算是 `stack`： （`stack` 和 `unstack` 将在本章后面详细讲解）

```
data.unstack().stack()

"""输出如下"""
a  1   -0.204708
   2    0.478943
   3   -0.519439
b  1   -0.555730
   3    1.965781
c  1    1.393406
   2    0.092908
d  2    0.281746
   3    0.769023
dtype: float64
```

对于一个 `DataFrame`， 每条轴都可以有分层索引

```
frame = pd.DataFrame(np.arange(12).reshape((4, 3)),
                     index=[['a', 'a', 'b', 'b'], [1, 2, 1, 2]],
                     columns=[['Ohio', 'Ohio', 'Colorado'],
                              ['Green', 'Red', 'Green']])
frame
```

		Ohio		Colorado
		Green	Red	Green
a	1	0	1	2
	2	3	4	5
b	1	6	7	8
	2	9	10	11

各层都可以有名字（可以是字符串，也可以是别的Python对象）。如果指定了名称，它们就会显示在控制台输出中：（注意：小心区分索引名 `state`、`color` 与行标签）

```
frame.index.names = ['key1', 'key2']
frame.columns.names = ['state', 'color']
frame
```

	state	Ohio		Colorado
	color	Green	Red	Green
key1	key2			
a	1	0	1	2
	2	3	4	5
b	1	6	7	8
	2	9	10	11

有了部分列索引，因此可以轻松选取列分组：

```
frame['Ohio']
```

	color	Green	Red
key1	key2		
a	1	0	1
	2	3	4
b	1	6	7
	2	9	10

可以单独创建 `MultiIndex` 然后复用。上面那个 `DataFrame` 中的（带有分级名称）列可以这样创建：

```
MultiIndex.from_arrays([['Ohio', 'Ohio', 'Colorado'], ['Green', 'Red', 'Green']],
                       names=['state', 'color'])

"""输出如下"""
MultiIndex(levels=[['Colorado', 'Ohio'], ['Green', 'Red']],
           labels=[[1, 1, 0], [0, 1, 0]],
           names=['state', 'color'])
```

12.2.2 重排与分级排序（Reordering and Sorting Levels）

- 有时，我们需要重新调整某条轴上各级别的顺序，或根据指定级别上的值对数据进行排序。`swaplevel` 接受两个级别编号或名称，并返回一个互换了级别的新对象（但数据不会发生变化）：

```
frame
```

	state	Ohio		Colorado
	color	Green	Red	Green
key1	key2			
a	1	0	1	2
	2	3	4	5
b	1	6	7	8
	2	9	10	11

- 互换级别

```
frame.swaplevel('key1', 'key2')
```

	state	Ohio		Colorado
	color	Green	Red	Green
key2	key1			
1	a	0	1	2
2	a	3	4	5
1	b	6	7	8
2	b	9	10	11

而 `sort_index` 则根据单个级别中的值对数据进行排序。交换级别时，常常也会用到 `sort_index`，这样最终结果就是按照指定顺序进行字母排序了：

```
# level=1, 按照key2排序
frame.sort_index(level=1)
```

	state	Ohio		Colorado
	color	Green	Red	Green
key1	key2			
a	1	0	1	2
b	1	6	7	8
a	2	3	4	5
b	2	9	10	11

先互换级别后为 `key2` 、 `key1` , `level=0` ,按照 `key2` 排序

```
frame.swaplevel(0, 1).sort_index(level=0)
```

	state	Ohio		Colorado
	color	Green	Red	Green
key2	key1			
1	a	0	1	2
	b	6	7	8
2	a	3	4	5
	b	9	10	11

12.2.3 根据级别汇总统计（Summary Statistics by Level）

- 许多对 `DataFrame` 和 `Series` 的描述和汇总统计都有一个level选项，它用于指定在某条轴上求和的级别。再以上面那个 `DataFrame` 为例，我们可以根据行或列上的级别来进行求和：

```
frame
```

	state	Ohio		Colorado
	color	Green	Red	Green
key1	key2			
a	1	0	1	2
	2	3	4	5
b	1	6	7	8
	2	9	10	11

```
frame.sum(level='key2')
```

state	Ohio		Colorado
color	Green	Red	Green
key2			
1	6	8	10
2	12	14	16

```
frame.sum(level='color', axis=1)
```

	color	Green	Red
key1	key2		
a	1	2	1
	2	8	4
b	1	14	7
	2	20	10

这其实是利用了 `pandas` 的 `groupby` 功能。

12.2.4 使用DataFrame的列进行索引(Indexing with a DataFrame's columns)

- 我们经常想要将 `DataFrame` 的一个或多个列当做行索引来用，或者可能希望将行索引变成DataFrame的列。以下面这个 `DataFrame` 为例：

```
frame = pd.DataFrame({'a': range(7), 'b': range(7, 0, -1),
                      'c': ['one', 'one', 'one', 'two', 'two',
                            'two', 'two'],
                      'd': [0, 1, 2, 0, 1, 2, 3]})
frame
```

	a	b	c	d
0	0	7	one	0
1	1	6	one	1
2	2	5	one	2
3	3	4	two	0
4	4	3	two	1
5	5	2	two	2
6	6	1	two	3

`DataFrame` 的 `set_index` 函数会将其一个或多个列转换为行索引，并创建一个新的 `DataFrame` 。

```
# 将c和d列设置为索引（multindex）
frame2 = frame.set_index(['c', 'd'])
frame2
```

		a	b
c	d		
one	0	0	7
	1	1	6
	2	2	5
two	0	3	4
	1	4	3
	2	5	2
	3	6	1

默认情况下，被设置为索引的列会从 `DataFrame` 中移除，但也可以将其保留下来：

```
# 默认情况下，被设置为索引的列会从DataFrame中移除，但也可以将其保留下来：
frame.set_index(['c', 'd'], drop=False)
```

		a	b	c	d
c	d				
one	0	0	7	one	0
	1	1	6	one	1
	2	2	5	one	2
two	0	3	4	two	0
	1	4	3	two	1
	2	5	2	two	2
	3	6	1	two	3

`reset_index` 的功能跟 `set_index` 刚好相反，层次化索引的级别会被转移到列里面：

```
frame2.reset_index()
```

	c	d	a	b
0	one	0	0	7
1	one	1	1	6
2	one	2	2	5
3	two	0	3	4
4	two	1	4	3
5	two	2	5	2
6	two	3	6	1

13 十二、Pandas怎样实现DataFrame的Merge

- `Pandas` 的 `Merge`，相当于 `Sql` 的 `Join`，将不同的表按 `key` 关联到一个表

13.1 merge的语法：

- 语法

```
pd.merge(left
         , right
         , how='inner'
         , on=None
         , left_on=None
         , right_on=None
         , left_index=False
         , right_index=False
         , sort=True
         , suffixes=('_x', '_y')
         , copy=True
         , indicator=False
         , validate=None)
```

1. `left`，`right`：要merge的dataframe或者有name的Series
2. `how`：join类型，'left', 'right', 'outer', 'inner'
3. `on`：join的key，left和right都需要有这个key
4. `left_on`：left的df或者series的key
5. `right_on`：right的df或者seires的key
6. `left_index`，right_index：使用index而不是普通的column做join
7. `suffixes`：两个元素的后缀，如果列有重名，自动添加后缀，默认是('x', 'y')

文档地址 https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.merge.htm

> **https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.merge.htm**

本次讲解提纲：

1. 电影数据集的join实例
2. 理解merge时一对一、一对多、多对多的数量对齐关系
3. 理解left join、right join、inner join、outer join的区别
4. 如果出现非Key的字段重名怎么办

13.2 电影数据集的join实例

13.2.1 电影评分数据集

- 是推荐系统研究的很好的数据集目录：./datas/movielens-1m

  包含三个文件：

  1. 用户对电影的评分数据 ratings.dat
  2. 用户本身的信息数据 users.dat
  3. 电影本身的数据 movies.dat

  可以关联三个表，得到一个完整的大表

  数据集官方地址

  > **https://grouplens.org/datasets/movielens/**

- 读取数据

```python
import pandas as pd

df_ratings = pd.read_csv(
    "./datas/movielens-1m/ratings.dat",
    sep="::",
    engine='python',
    names="UserID::MovieID::Rating::Timestamp".split("::")
)

df_ratings.head()
```

	UserID	MovieID	Rating	Timestamp
0	1	1193	5	978300760
1	1	661	3	978302109
2	1	914	3	978301968
3	1	3408	4	978300275
4	1	2355	5	978824291

```python
df_users = pd.read_csv(
    "./datas/movielens-1m/users.dat",
    sep="::",
    engine='python',
    names="UserID::Gender::Age::Occupation::Zip-code".split("::")
)

df_users.head()
```

	UserID	Gender	Age	Occupation	Zip-code
0	1	F	1	10	48067
1	2	M	56	16	70072
2	3	M	25	15	55117
3	4	M	45	7	02460
4	5	M	25	20	55455

```
df_movies = pd.read_csv(
    "./datas/movielens-1m/movies.dat",
    sep="::",
    engine='python',
    names="MovieID::Title::Genres".split("::")
)

df_movies.head()
```

	MovieID	Title	Genres
0	1	Toy Story (1995)	Animation|Children's|Comedy
1	2	Jumanji (1995)	Adventure|Children's|Fantasy
2	3	Grumpier Old Men (1995)	Comedy|Romance
3	4	Waiting to Exhale (1995)	Comedy|Drama
4	5	Father of the Bride Part II (1995)	Comedy

```
df_ratings_users = pd.merge(
 df_ratings,df_users,left_on='UserID',right_on='UserID',how='inner'
)

df_ratings_users.head()
```

	UserID	MovieID	Rating	Timestamp	Gender	Age	Occupation	Zip-code
0	1	1193	5	978300760	F	1	10	48067
1	1	661	3	978302109	F	1	10	48067
2	1	914	3	978301968	F	1	10	48067
3	1	3408	4	978300275	F	1	10	48067
4	1	2355	5	978824291	F	1	10	48067

```
df_ratings_users_movies = pd.merge(
    df_ratings_users, df_movies, left_on="MovieID",
right_on="MovieID", how="inner"
)

df_ratings_users_movies.head()
```

13.2.2 理解merge时数量的对齐关系

- 以下关系要正确理解：
- one-to-one：一对一关系，关联的key都是唯一的
    - 比如(学号，姓名) merge (学号，年龄)
    - 结果条数为：1 VS 1
- one-to-many：一对多关系，左边唯一key，右边不唯一key
    - 比如(学号，姓名) merge (学号，[语文成绩、数学成绩、英语成绩])
    - 结果条数为：1 VS N
- many-to-many：多对多关系，左边右边都不是唯一的
    - 比如（学号，[语文成绩、数学成绩、英语成绩]） merge (学号，[篮球、足球、乒乓球])
    - 结果条数为：M*N

13.2.2.1 one-to-one

- 一对一关系的merge

```
left = pd.DataFrame({'sno': [11, 12, 13, 14],
                     'name': ['name_a', 'name_b', 'name_c', 'name_d']
                    })
left
```

	sno	name
0	11	name_a
1	12	name_b
2	13	name_c
3	14	name_d

```python
right = pd.DataFrame({'sno': [11, 12, 13, 14],
                      'age': ['21', '22', '23', '24']
                     })
right
```

	sno	age
0	11	21
1	12	22
2	13	23
3	14	24

一对一关系，结果中有4条

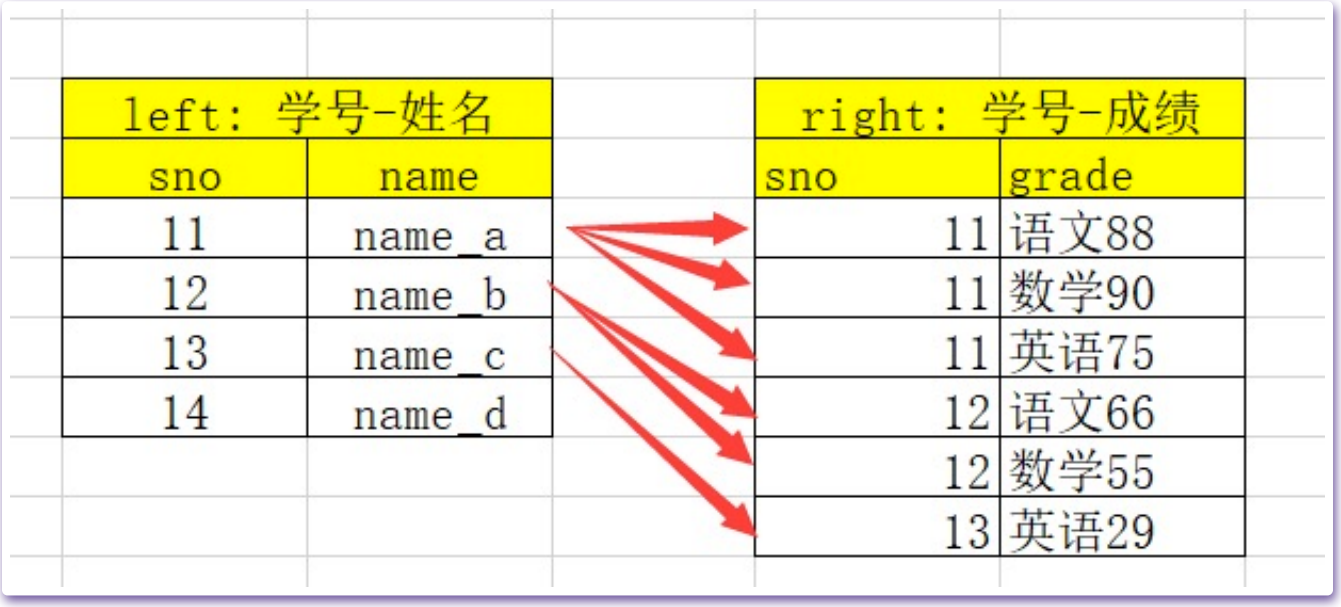
```python
# 一对一关系，结果中有4条
pd.merge(left,right,on='sno')

"""或者"""

# 一对一关系，结果中有4条
pd.merge(left,right,left_on='sno',right_on='sno',how='inner')
```

	sno	name	age
0	11	name_a	21
1	12	name_b	22
2	13	name_c	23
3	14	name_d	24

13.2.2.2 one-to-many 一对多关系的merge



- one-to-many 一对多关系的merge

注意：数据会被复制

```
left = pd.DataFrame({'sno': [11, 12, 13, 14],
                     'name': ['name_a', 'name_b', 'name_c', 'name_d']
                    })
left
```

	sno	name
0	11	name_a
1	12	name_b
2	13	name_c
3	14	name_d

```
right = pd.DataFrame({'sno': [11, 11, 11, 12, 12, 13],
                      'grade': ['语文88', '数学90', '英语75','语文66', '数学55', '英语29']
                     })
right
```

	sno	grade
0	11	语文88
1	11	数学90
2	11	英语75
3	12	语文66
4	12	数学55
5	13	英语29

```
# 数目以多的一边为准
pd.merge(left,right,on='sno')
```

	sno	name	grade
0	11	name_a	语文88
1	11	name_a	数学90
2	11	name_a	英语75
3	12	name_b	语文66
4	12	name_b	数学55
5	13	name_c	英语29

13.2.2.3 many-to-many

- many-to-many 多对多关系的merge

  注意：结果数量会出现乘法

```
left = pd.DataFrame({'sno': [11, 11, 12, 12,12],
                     '爱好': ['篮球', '羽毛球', '乒乓球', '篮球', "足球"]
                    })
left
```

	sno	爱好
0	11	篮球
1	11	羽毛球
2	12	乒乓球
3	12	篮球
4	12	足球

```
right = pd.DataFrame({'sno': [11, 11, 11, 12, 12, 13],
                      'grade': ['语文88', '数学90', '英语75','语文66', '数学55', '英语29']
                     })
right
```

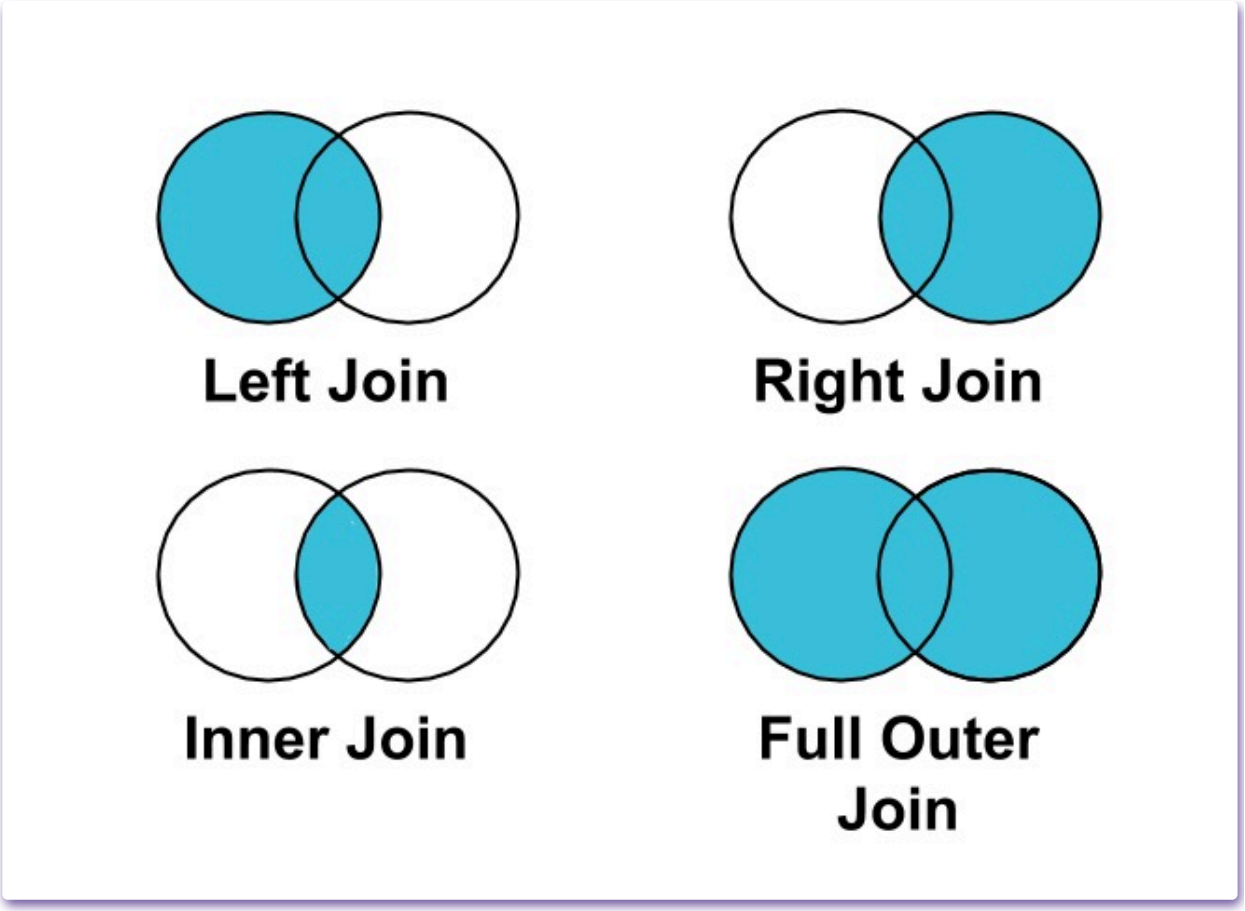
	sno	grade
0	11	语文88
1	11	数学90
2	11	英语75
3	12	语文66
4	12	数学55
5	13	英语29

```
pd.merge(left, right, on='sno')
```

	sno	爱好	grade
0	11	篮球	语文88
1	11	篮球	数学90
2	11	篮球	英语75
3	11	羽毛球	语文88
4	11	羽毛球	数学90
5	11	羽毛球	英语75
6	12	乒乓球	语文66
7	12	乒乓球	数学55
8	12	篮球	语文66
9	12	篮球	数学55
10	12	足球	语文66
11	12	足球	数学55

13.2.3 理解join方法的区别

- 理解left join、right join、inner join、outer join的区别




```
left = pd.DataFrame({'key': ['K0', 'K1', 'K2', 'K3'],
                     'A': ['A0', 'A1', 'A2', 'A3'],
                     'B': ['B0', 'B1', 'B2', 'B3']})

right = pd.DataFrame({'key': ['K0', 'K1', 'K4', 'K5'],
                      'C': ['C0', 'C1', 'C4', 'C5'],
                      'D': ['D0', 'D1', 'D4', 'D5']})
```

```
left
```

	key	A	B
0	K0	A0	B0
1	K1	A1	B1
2	K2	A2	B2
3	K3	A3	B3

```
right
```

	key	C	D
0	K0	C0	D0
1	K1	C1	D1
2	K4	C4	D4
3	K5	C5	D5

13.2.3.1 inner join(默认)

- `inner join`，默认

  左边和右边的key都有，才会出现在结果里

```
pd.merge(left, right, how='inner')
```

	key	A	B	C	D
0	K0	A0	B0	C0	D0
1	K1	A1	B1	C1	D1

13.2.3.2 left join

- 左边的都会出现在结果里，右边的如果无法匹配则为 `Null`

```
pd.merge(left, right, how='left')
```

	key	A	B	C	D
0	K0	A0	B0	C0	D0
1	K1	A1	B1	C1	D1
2	K2	A2	B2	NaN	NaN
3	K3	A3	B3	NaN	NaN

13.2.3.3 right join

- 右边的都会出现在结果里，左边的如果无法匹配则为 `Null`

```
pd.merge(left, right, how='right')
```

	key	A	B	C	D
0	K0	A0	B0	C0	D0
1	K1	A1	B1	C1	D1
2	K4	NaN	NaN	C4	D4
3	K5	NaN	NaN	C5	D5

13.2.3.4 outer join

- 左边、右边的都会出现在结果里，如果无法匹配则为 `Null`

```
pd.merge(left, right, how='outer')
```

	key	A	B	C	D
0	K0	A0	B0	C0	D0
1	K1	A1	B1	C1	D1
2	K2	A2	B2	NaN	NaN
3	K3	A3	B3	NaN	NaN
4	K4	NaN	NaN	C4	D4
5	K5	NaN	NaN	C5	D5

13.2.4 非Key字段重名

- 如果出现非Key的字段重名怎么办

```
left = pd.DataFrame({'key': ['K0', 'K1', 'K2', 'K3'],
                     'A': ['A0', 'A1', 'A2', 'A3'],
                     'B': ['B0', 'B1', 'B2', 'B3']})

right = pd.DataFrame({'key': ['K0', 'K1', 'K4', 'K5'],
                      'A': ['A10', 'A11', 'A12', 'A13'],
                      'D': ['D0', 'D1', 'D4', 'D5']})
```

```
left
```

	key	A	B
0	K0	A0	B0
1	K1	A1	B1
2	K2	A2	B2
3	K3	A3	B3

```
right
```

	key	A	D
0	K0	A10	D0
1	K1	A11	D1
2	K4	A12	D4
3	K5	A13	D5

```
pd.merge(left, right, on='key')
```

	key	A_x	B	A_y	D
0	K0	A0	B0	A10	D0
1	K1	A1	B1	A11	D1

`suffixes`

```
# suffixes：两个元素的后缀，如果列有重名，自动添加后缀，默认是('_x', '_y')
pd.merge(left, right, on='key', suffixes=('_left', '_right'))
```

	key	A_left	B	A_right	D
0	K0	A0	B0	A10	D0
1	K1	A1	B1	A11	D1

13.3 索引上的合并（Merging on Index）

- 有时候，`DataFrame` 中的连接键位于其索引中。在这种情况下，你可以传入 `left_index=True` 或 `right_index=True`（或两个都传）以说明索引应该被用作连接键：

```
left1 = pd.DataFrame({'key': ['a', 'b', 'a', 'a', 'b', 'c'],
                      'value': range(6)})
right1 = pd.DataFrame({'group_val': [3.5, 7]}, index=['a', 'b'])
```

```
left1
```

	key	value
0	a	0
1	b	1
2	a	2
3	a	3
4	b	4
5	c	5

```
right1
```

	group_val
a	3.5
b	7.0

默认为交集

```
# 默认为交集
pd.merge(left1, right1, left_on='key', right_index=True)
```

	key	value	group_val
0	a	0	3.5
2	a	2	3.5
3	a	3	3.5
1	b	1	7.0
4	b	4	7.0

由于默认的 `merge` 方法是求取连接键的交集，因此你可以通过外连接的方式得到它们的并集：

```
pd.merge(left1, right1, left_on='key', right_index=True,
how='outer')
```

	key	value	group_val
0	a	0	3.5
2	a	2	3.5
3	a	3	3.5
1	b	1	7.0
4	b	4	7.0
5	c	5	NaN

- 对于层次化索引的数据，事情就有点复杂了，因为索引的合并默认是多键合并：

```
lefth = pd.DataFrame({'key1': ['Ohio', 'Ohio', 'Ohio',
                               'Nevada', 'Nevada'],
                      'key2': [2000, 2001, 2002, 2001, 2002],
                      'data': np.arange(5.)})
righth = pd.DataFrame(np.arange(12).reshape((6, 2)),
                      index=[['Nevada', 'Nevada', 'Ohio', 'Ohio',
                              'Ohio', 'Ohio'],
                             [2001, 2000, 2000, 2000, 2001, 2002]],
                      columns=['event1', 'event2'])
```

```
lefth
```

	key1	key2	data
0	Ohio	2000	0.0
1	Ohio	2001	1.0
2	Ohio	2002	2.0
3	Nevada	2001	3.0
4	Nevada	2002	4.0

```
righth
```

		event1	event2
Nevada	2001	0	1
	2000	2	3
Ohio	2000	4	5
	2000	6	7
	2001	8	9
	2002	10	11

这种情况下，你必须以列表的形式指明用作合并键的多个列（注意用 `how='outer'` 对重复索引值的处理）：

```
# 默认为交集
pd.merge(lefth, righth, left_on=['key1', 'key2'], right_index=True)
```

	key1	key2	data	event1	event2
0	Ohio	2000	0.0	4	5
0	Ohio	2000	0.0	6	7
1	Ohio	2001	1.0	8	9
2	Ohio	2002	2.0	10	11
3	Nevada	2001	3.0	0	1

```
# 并集
pd.merge(lefth, righth, left_on=['key1', 'key2'],
         right_index=True, how='outer')
```

	key1	key2	data	event1	event2
0	Ohio	2000	0.0	4.0	5.0
0	Ohio	2000	0.0	6.0	7.0
1	Ohio	2001	1.0	8.0	9.0
2	Ohio	2002	2.0	10.0	11.0
3	Nevada	2001	3.0	0.0	1.0
4	Nevada	2002	4.0	NaN	NaN
4	Nevada	2000	NaN	2.0	3.0

- 同时使用合并双方的索引也没问题：

```
left2 = pd.DataFrame([[1., 2.], [3., 4.], [5., 6.]],
                     index=['a', 'c', 'e'],
                     columns=['Ohio', 'Nevada'])
right2 = pd.DataFrame([[7., 8.], [9., 10.], [11., 12.], [13, 14]],
                      index=['b', 'c', 'd', 'e'],
                      columns=['Missouri', 'Alabama'])
```

```
left2
```

	Ohio	Nevada
a	1.0	2.0
c	3.0	4.0
e	5.0	6.0

```
right2
```

	Missouri	Alabama
b	7.0	8.0
c	9.0	10.0
d	11.0	12.0
e	13.0	14.0

```
pd.merge(left2, right2, how='outer', left_index=True,
right_index=True)
```

	Ohio	Nevada	Missouri	Alabama
a	1.0	2.0	NaN	NaN
b	NaN	NaN	7.0	8.0
c	3.0	4.0	9.0	10.0
d	NaN	NaN	11.0	12.0
e	5.0	6.0	13.0	14.0

- `DataFrame` 还有一个便捷的 `join` 实例方法，它能更为方便地实现按索引合并。它还可用于合并多个带有相同或相似索引的 `DataFrame` 对象，但要求没有重叠的列。在上面那个例子中，我们可以编写：

```
left2.join(right2, how='outer')
```

	Ohio	Nevada	Missouri	Alabama
a	1.0	2.0	NaN	NaN
b	NaN	NaN	7.0	8.0
c	3.0	4.0	9.0	10.0
d	NaN	NaN	11.0	12.0
e	5.0	6.0	13.0	14.0

因为一些历史版本的遗留原因， `DataFrame` 的 `join` 方法默认使用的是左连接，保留左边表的行索引。它还支持在调用的 `DataFrame` 的列上，连接传递的 `DataFrame` 索引

```
left1.join(right1, on='key')
```

	key	value	group_val
0	a	0	3.5
1	b	1	7.0
2	a	2	3.5
3	a	3	3.5
4	b	4	7.0
5	c	5	NaN

最后，对于简单的索引合并，你还可以向 `join` 传入一组 `DataFrame`，下一节会介绍更为通用的 `concat` 函数，也能实现此功能：

```
another = pd.DataFrame([[7., 8.], [9., 10.], [11., 12.], [16., 17.]],
                       index=['a', 'c', 'e', 'f'],
                       columns=['New York', 'Oregon'])
another
left2.join([right2, another])
left2.join([right2, another], how='outer')
```

14 十三、Pandas实现数据的合并concat

- 使用场景：
    1. 批量合并相同格式的Excel；
    2. 给 `DataFrame` 添加行；
    3. 给 `DataFrame` 添加列。
- 一句话说明 `concat` 语法：
    1. 使用某种合并方式(inner/outer)
    2. 沿着某个轴向(axis=0/1)
    3. 把多个Pandas对象(DataFrame/Series)合并成一个。
- `concat` 语法：

```
pandas.concat(objs
              , axis=0
              , join='outer'
              , ignore_index=False
             )
```

    1. `objs`： 一个列表，内容可以是DataFrame或者Series，可以混合
    2. `axis`： 默认是0代表按行合并，如果等于1代表按列合并
    3. `join`： 合并的时候索引的对齐方式，默认是outer join，也可以是inner join
    4. `ignore_index`： 是否忽略掉原来的数据索引
- `append` 语法：

```
DataFrame.append(other
                 , ignore_index=False
                )
```

`append只有按行合并，没有按列合并，相当于concat按行的简写形式`

    1. `other`： 单个 `dataframe` 、 `series` 、 `dict` ，或者列表
    2. `ignore_index`： 是否忽略掉原来的数据索引
- 参考文档：
    1. **pandas.concat的api文档**：

    > **https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.concat.html**

    2. **pandas.concat的教程**：

    > **https://pandas.pydata.org/pandas-docs/stable/user_guide/merging.html**

3. **pandas.append的api文档**：

> **https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.append.html**

14.1 pandas.concat

- 使用pandas.concat合并数据

```
df1 = pd.DataFrame({'A': ['A0', 'A1', 'A2', 'A3'],
                    'B': ['B0', 'B1', 'B2', 'B3'],
                    'C': ['C0', 'C1', 'C2', 'C3'],
                    'D': ['D0', 'D1', 'D2', 'D3'],
                    'E': ['E0', 'E1', 'E2', 'E3']
                   })
df1
```

```
df2 = pd.DataFrame({ 'A': ['A4', 'A5', 'A6', 'A7'],
                     'B': ['B4', 'B5', 'B6', 'B7'],
                     'C': ['C4', 'C5', 'C6', 'C7'],
                     'D': ['D4', 'D5', 'D6', 'D7'],
                     'F': ['F4', 'F5', 'F6', 'F7']
                   })
df2
```

14.1.1 默认

- 默认的 `concat`，参数为 `axis` =0、 `join` =outer、 `ignore_index` =False

```
pd.concat([df1,df2])
```

14.1.2 参数指定

14.1.2.1 ignore_index

- 使用 `ignore_index` =True可以忽略原来的索引

```
pd.concat([df1,df2], ignore_index=True)
```

14.1.2.2 join

- 使用 `join` =inner过滤掉不匹配的列

```
pd.concat([df1,df2], ignore_index=True, join="inner")
```

14.1.2.3 axis

- 使用axis=1相当于添加新列

```
df1
```

- 添加一列 `Series`

```
s1 = pd.Series(list(range(4)), name="F1")
pd.concat([df1,s1], axis=1)
```

- 添加多列 `Series`

```
s2 = df1.apply(lambda x:x["A"]+"_GG", axis=1)
s2
s2.name='G'

"""输出如下"""
0    A0_GG
1    A1_GG
2    A2_GG
3    A3_GG
dtype: object
```

```
# s1 上同时添加两列
pd.concat([df1,s1,s2], axis=1)
```

传入列表中可以只有 `Series`

```
# 列表中可以只有Series， 如s1、s2都是series
pd.concat([s1,s2], axis=1)
```

传入列表 `series` 、 `dataframe` 可以混合顺序的

```
# 列表是可以混合顺序的
pd.concat([s1,df1,s2], axis=1)
```

14.1.3 轴向连接（Concatenating Along an Axis)

- 连接（`concatenation`）,也被称作,绑定（`binding`）或堆叠（`stacking`）。`NumPy`的`concatenation`函数可以用`NumPy`数组来做：

```
arr = np.arange(12).reshape((3, 4))
arr


"""输出如下"""
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

```
np.concatenate([arr, arr], axis=1)


"""输出如下"""
array([[ 0,  1,  2,  3,  0,  1,  2,  3],
       [ 4,  5,  6,  7,  4,  5,  6,  7],
       [ 8,  9, 10, 11,  8,  9, 10, 11]])
```

假设有三个没有重叠索引的`index`

```
# 假设有三个没有重叠索引的index
s1 = pd.Series([0, 1], index=['a', 'b'])
s2 = pd.Series([2, 3, 4], index=['c', 'd', 'e'])
s3 = pd.Series([5, 6], index=['f', 'g'])
```

对这些对象调用`concat`可以将值和索引粘合在一起

```
pd.concat([s1, s2, s3])


"""输出如下"""
a    0
b    1
c    2
d    3
e    4
f    5
g    6
dtype: int64
```

- 默认情况下，`concat`是在`axis=0`上工作的，最终产生一个新的`Series`。

```
# 默认情况下，concat是在axis=0上工作的，最终产生一个新的Series。如果传入axis=1，则结果就会变成一个DataFrame（axis=1是列）：

pd.concat([s1, s2, s3], axis=1)
```

这种情况下，另外的轴上没有重叠，从索引的有序并集（外连接）上就可以看出来。传入`join='inner'`即可得到它们的交集：

```
s4 = pd.concat([s1, s3])
s4


"""输出如下"""
a    0
b    1
f    5
g    6
dtype: int64
```

```
pd.concat([s1, s4], axis=1)
```

```
pd.concat([s1, s4], axis=1, join='inner')
```

假设你想要在连接轴上创建一个层次化索引。使用 `keys` 参数即可达到这个目的：

```
result = pd.concat([s1, s1, s3], keys=['one', 'two', 'three'])
result


"""输出如下"""
one    a    0
       b    1
two    a    0
       b    1
three  f    5
       g    6
dtype: int64
```

```
result.unstack()
```

如果沿着 `axis=1` 对 `Series` 进行合并，则 `keys` 就会成为 `DataFrame` 的列头：

```
pd.concat([s1, s2, s3], axis=1, keys=['one', 'two', 'three'])
```

- 同样的逻辑也适用于 `DataFrame` 对象：

```
df1 = pd.DataFrame(np.arange(6).reshape(3, 2), index=['a', 'b', 'c'],
                   columns=['one', 'two'])
df2 = pd.DataFrame(5 + np.arange(4).reshape(2, 2), index=['a', 'c'],
                   columns=['three', 'four'])
print(df1)
print(df2)
pd.concat([df1, df2], axis=1, keys=['level1', 'level2'])

```

```

"""输出如下"""
   one  two
a    0    1
b    2    3
c    4    5


   three  four
a      5     6
c      7     8
```

如果传入的不是列表而是一个字典，则字典的键就会被当做 `keys` 选项的值：

```
pd.concat({'level1': df1, 'level2': df2}, axis=1)
```

我们可以用 `names` 参数命名创建的轴级别

```
pd.concat([df1, df2], axis=1, keys=['level1', 'level2'],
          names=['upper', 'lower'])
```

最后一个关于 `DataFrame` 的问题是， `DataFrame` 的行索引不包含任何相关数据：

```
df1 = pd.DataFrame(np.random.randn(3, 4), columns=['a', 'b', 'c', 'd'])
df2 = pd.DataFrame(np.random.randn(2, 3), columns=['b', 'd', 'a'])
print(df1)
print(df2)


"""输出如下"""
          a         b         c         d
0  1.246435  1.007189 -1.296221  0.274992
1  0.228913  1.352917  0.886429 -2.001637
2 -0.371843  1.669025 -0.438570 -0.539741


          b         d         a
0  0.476985  3.248944 -1.021228
1 -0.577087  0.124121  0.302614
pd.concat([df1, df2], ignore_index=True)
```

```
pd.concat([df1, df2], ignore_index=True)
```

14.2 DataFrame.append

- 使用 `DataFrame.append` 按行合并数据

```
df1 = pd.DataFrame([[1, 2], [3, 4]], columns=list('AB'))
df1
```

```
df2 = pd.DataFrame([[5, 6], [7, 8]], columns=list('AB'))
df2
```

14.2.1 给1个dataframe添加另一个dataframe

- 给1个 `dataframe` 添加另一个 `dataframe`

```
df1.append(df2)
```

- 忽略原来的索引 `ignore_index` =True

```
df1.append(df2,ignore_index=True)
```

14.2.2 一行一行的给DataFrame添加数据

- 可以一行一行的给DataFrame添加数据

```
# 一个空的df
df = pd.DataFrame(columns=['A'])
df
```

- A：低性能版本

```
for i in range(5):
    # 注意这里每次都在复制
    df = df.append({'A':i}, ignore_index=True)
df
```

- B：高性能版本

```
# 第一个入参是一个列表，避免了多次复制
pd.concat(
    [pd.DataFrame([i], columns=['A']) for i in range(5)],
    ignore_index=True
)
```

15 十四、Pandas重塑和轴向旋转

- 有许多用于重新排列表格型数据的基础运算。这些函数也称作重 塑（`reshape`）或轴向旋转（`pivot`）运算。

15.1 重塑层次化索引

- 重塑层次化索引(Reshaping with Hierarchical Indexing)

  层次化索引为`DataFrame`数据的重排任务提供了一种具有良好一 致性的方式。主要功能有二：

  1. `stack`：将数据的列“旋转”为行。
  2. `unstack`：将数据的行“旋转”为列。

- 接下来看一个简单的`DataFrame`，其中的行列索引均为字符串数组

```
data = pd.DataFrame(np.arange(6).reshape((2, 3)),
                    index=pd.Index(['Ohio', 'Colorado'], name='state'),
                    columns=pd.Index(['one', 'two', 'three'],
                    name='number'))
data
```

对该数据使用`stack`方法即可将列转换为行，得到一个`Series`：

```
result = data.stack()
result


"""输出如下"""
state     number
Ohio      one       0
          two       1
          three     2
Colorado  one       3
          two       4
          three     5
dtype: int64
```

对于一个层次化索引的`Series`，你可以用`unstack`将其重排为一 个`DataFrame`：

```
result.unstack()
```

默认情况下，`unstack`操作的是最内层（`stack`也是如此）。传入 分层级别的编号或名称即可对其它级别进行unstack操作：

```
result.unstack(0)
```

```
result.unstack('state')
```

- 如果不是所有的级别值都能在各分组中找到的话，则unstack操 作可能会引入缺失数据：

```
s1 = pd.Series([0, 1, 2, 3], index=['a', 'b', 'c', 'd'])
s2 = pd.Series([4, 5, 6], index=['c', 'd', 'e'])
data2 = pd.concat([s1, s2], keys=['one', 'two'])
print(data2)
data2.unstack()


"""输出如下"""
one  a    0
     b    1
     c    2
     d    3
two  c    4
     d    5
     e    6
dtype: int64
```

stack默认会滤除缺失数据，因此该运算是可逆的：

```
print(data2.unstack())
print()
print(data2.unstack().stack())
print()
print(data2.unstack().stack(dropna=False))



"""输出如下"""
        a    b    c    d    e
one   0.0  1.0  2.0  3.0  NaN
two   NaN  NaN  4.0  5.0  6.0

one  a    0.0
     b    1.0
     c    2.0
     d    3.0
two  c    4.0
     d    5.0
     e    6.0
dtype: float64

one  a    0.0
     b    1.0
     c    2.0
     d    3.0
```

```
        e    NaN
two     a    NaN
        b    NaN
        c    4.0
        d    5.0
        e    6.0
dtype: float64
```

- 在对 `DataFrame` 进行 `unstack` 操作时，作为旋转轴的级别将会成 为结果中的最低级别：

```
df = pd.DataFrame({'left': result, 'right': result + 5},
                  columns=pd.Index(['left', 'right'], name='side'))
print(df)
print()
df.unstack('state')


"""输出如下"""
side              left  right
state    number
Ohio     one         0      5
         two         1      6
         three       2      7
Colorado one         3      8
         two         4      9
         three       5     10
```

当调用 `stack`，我们可以指明轴的名字：

```
df.unstack('state').stack('side')
```

15.2 将“长格式”旋转为“宽格式”

- 将“长格式”旋转为“宽格式”(Pivoting “Long” to “Wide” Format)

  多个时间序列数据通常是以所谓的“长格式”（long）或“堆叠格式”（stacked）存储在数据库和CSV中的。我们先加载一些示例数据，做一些时间序列规整和数据清洗：

```
data = pd.read_csv('examples/macrodata.csv')
data.head()
```

```
periods = pd.PeriodIndex(year=data.year
                         , quarter=data.quarter
                         , name='date')
periods


"""输出如下"""
PeriodIndex(['1959Q1', '1959Q2', '1959Q3', '1959Q4', '1960Q1', '1960Q2',
             '1960Q3', '1960Q4', '1961Q1', '1961Q2',
             ...
             '2007Q2', '2007Q3', '2007Q4', '2008Q1', '2008Q2', '2008Q3',
             '2008Q4', '2009Q1', '2009Q2', '2009Q3'],
            dtype='period[Q-DEC]', name='date', length=203, freq='Q-DEC')
```

```
columns = pd.Index(['realgdp', 'infl', 'unemp'], name='item')
columns


Index(['realgdp', 'infl', 'unemp'], dtype='object', name='item')
```

```
data = data.reindex(columns=columns)
data
```

```
data.index = periods.to_timestamp('D'
                                  , 'end')
data
```

```
ldata = data.stack().reset_index().rename(columns={0: 'value'})
ldata[:10]
```

- 上面的就是多个时间序列（或者其它带有两个或多个键的可观察数 据，这里，我们的键是 `date` 和 `item` ）的长格式。表中的每行代表 一次观察。

  关系型数据库（如MySQL）中的数据经常都是这样存储的，因 为固定架构（即列名和数据类型）有一个好处：随着表中数据的 添加，item列中的值的种类能够增加。在前面的例子中，date和 item通常就是主键（用关系型数据库的说法），不仅提供了关系 完整性，而且提供了更为简单的查询支持。有的情况下，使用这 样的数据会很麻烦，你可能会更喜欢 `DataFrame` ，不同的item值 分别形成一列，date列中的时间戳则用作索引。 `DataFrame` 的 `pivot` 方法完全可以实现这个转换：

  前两个传递的值分别用作行和列索引，最后一个可选值则是用于 填充 `DataFrame` 的数据列。假设有两个需要同时重塑的数据列：

```
pivoted = ldata.pivot('date' # 行索引
                      , 'item' # 列索引
                      , 'value' # 可选值，填充DataFrame数据列
                      )
pivoted
```

前两个传递的值分别用作行和列索引，最后一个可选值则是用于 填充 `DataFrame` 的数据列。假设有两个需要同时重塑的数据列：

```
ldata['value2'] = np.random.randn(len(ldata))
ldata[:10]
```

如果忽略最后一个参数，得到的 `DataFrame` 就会带有层次化的 列：

```
pivoted = ldata.pivot('date', 'item')
pivoted[:5]
```

```
pivoted['value'][:5]
```

```
pivoted['value2'][:5]
```

- 注意， `pivot` 其实就是用 `set_index` 创建层次化索引，再用 `unstack` 重塑：

```
ldata.set_index(['date', 'item'])
```

```
unstacked = ldata.set_index(['date', 'item']).unstack('item')
unstacked[:7]
```

15.3 将“宽格式”旋转为“长格式”

- 旋转 `DataFrame` 的逆运算是 `pandas.melt` 。它不是将一列转换到 多个新的 `DataFrame` ， 而是合并多个列成为一个，产生一个比输 入长的 `DataFrame` 。看一个例子：

```
df = pd.DataFrame({'key': ['foo', 'bar', 'baz'],
                   'A': [1, 2, 3],
                   'B': [4, 5, 6],
                   'C': [7, 8, 9]})
df
```

`key` 列可能是分组指标，其它的列是数据值。当使用 `pandas.melt` ， 我们必须指明哪些列是分组指标。下面使用 `key` 作 为唯一的分组指标：

```
melted = pd.melt(df, ['key'])
melted
```

使用 `pivot`，可以重塑回原来的样子：

```
reshaped = melted.pivot('key', 'variable', 'value')
reshaped
```

因为 `pivot` 的结果从列创建了一个索引，用作行标签，我们可以使用 `reset_index` 将数据移回列：

```
reshaped.reset_index()
```

可以指定列的子集，作为值的列：

```
pd.melt(df, id_vars=['key'], value_vars=['A', 'B'])
```

`pandas.melt` 也可以不用分组指标：

```
df
```

```
pd.melt(df, value_vars=['A', 'B', 'C'])
```

```
pd.melt(df, value_vars=['key', 'A', 'B'])
```

16 十五、Pandas怎样实现groupby分组统计

- 类似SQL：

> select city,max(temperature) from city_weather group by city;

`groupby`：先对数据分组，然后在每个分组上应用聚合函数、转换函数

本次演示：

1. 分组使用聚合函数做数据统计
2. 遍历 `groupby` 的结果理解执行流程
3. 实例分组探索天气数据

```
# 加上这一句，能在jupyter notebook展示matplot图表
%matplotlib inline
```

```
df = pd.DataFrame({'A': ['foo', 'bar', 'foo', 'bar', 'foo', 'bar', 'foo', 'foo'],
                   'B': ['one', 'one', 'two', 'three', 'two', 'two', 'one', 'three'],
                   'C': np.random.randn(8),
                   'D': np.random.randn(8)})
df
```

16.1 分组使用聚合函数做数据统计

16.1.1 单个列groupby

- 单个列 `groupby`，查询所有数据列的统计

```
df.groupby('A').sum()
```

我们看到：

1. `groupby` 中的'A'变成了数据的索引列
2. 因为要统计 `sum`，但B列不是数字，所以被自动忽略掉

16.1.2 多个列groupby

- 多个列 `groupby`，查询所有数据列的统计

```
df.groupby(['A','B']).mean()
```

我们看到：('A','B')成对变成了二级索引

可以设置 `as_index`

```
# as_index= False 不变为索引
df.groupby(['A','B'], as_index=False).mean()
```

16.1.3 同时查看多种数据统计

- 同时查看多种数据统计

```
df.groupby('A').agg([np.sum, np.mean, np.std])
```

我们看到：列变成了多级索引

16.1.4 查看单列的结果数据统计

- 方法1：预过滤，性能更好

```
# 方法1：预过滤，性能更好
df.groupby('A')['C'].agg([np.sum,np.mean,np.std])
```

- 方法2：

```
# 方法2：
df.groupby('A').agg([np.sum,np.mean,np.std])['C']
```

16.1.5 不同列使用不同的聚合函数

- 不同列使用不同的聚合函数

```
df.groupby('A').agg({'C':np.sum,'D':np.mean})
```

16.2 遍历groupby的结果理解执行流程

- `for` 循环可以直接遍历每个 `group`

16.2.1 遍历单个列聚合的分组

- 遍历单个列聚合的分组

```
g = df.groupby('A')
g


"""输出如下"""
<pandas.core.groupby.generic.DataFrameGroupBy object at 0x12da590b8>
```

```
for name,group in g:
    print(name)
    print(group)
    print()


"""输出如下"""
bar
     A      B         C         D
1  bar    one -0.498624  0.889612
3  bar  three -0.225370  1.531037
5  bar    two -0.510594  1.349258

foo
     A      B         C         D
0  foo    one -1.705063  1.043028
2  foo    two -0.376724 -0.531354
4  foo    two -1.211737 -1.542968
6  foo    one  0.887606  2.856420
7  foo  three  1.410297  0.14916
```

可以获取单个分组的数据

```
g.get_group('bar')
```

16.2.2 遍历多个列聚合的分组

- 遍历多个列聚合的分组

```
g = df.groupby(['A', 'B'])
g


"""输出如下"""
<pandas.core.groupby.groupby.DataFrameGroupBy object at 0x11baf02b0>
```

```
for name,group in g:
    print(name)
    print(group)
    print()


"""输出如下"""
('bar', 'one')
```

```
     A    B         C         D
1  bar  one -0.498624  0.889612

('bar', 'three')
     A      B        C         D
3  bar  three -0.22537  1.531037

('bar', 'two')
     A    B         C         D
5  bar  two -0.510594  1.349258

('foo', 'one')
     A    B         C         D
0  foo  one -1.705063  1.043028
6  foo  one  0.887606  2.856420

('foo', 'three')
     A      B         C         D
7  foo  three  1.410297  0.149169

('foo', 'two')
     A    B         C         D
2  foo  two -0.376724 -0.531354
4  foo  two -1.211737 -1.542968
```

可以看到，`name`是一个2个元素的`tuple`，代表不同的列

```
g.get_group(('foo', 'one'))
```

可以直接查询`group`后的某几列，生成Series`或者`子`DataFrame`

```
g['C']


"""输出如下"""
<pandas.core.groupby.generic.SeriesGroupBy object at 0x12da32cf8>
```

```
for name, group in g['C']:
    print(name)
    print(group)
    print(type(group))
    print()


"""输出如下"""
('bar', 'one')
1   -0.498624
Name: C, dtype: float64
<class 'pandas.core.series.Series'>

('bar', 'three')
```

```
3   -0.22537
Name: C, dtype: float64
<class 'pandas.core.series.Series'>

('bar', 'two')
5   -0.510594
Name: C, dtype: float64
<class 'pandas.core.series.Series'>

('foo', 'one')
0   -1.705063
6    0.887606
Name: C, dtype: float64
<class 'pandas.core.series.Series'>

('foo', 'three')
7    1.410297
Name: C, dtype: float64
<class 'pandas.core.series.Series'>

('foo', 'two')
2   -0.376724
4   -1.211737
Name: C, dtype: float64
<class 'pandas.core.series.Series'>
```

其实所有的聚合统计，都是在dataframe和series上进行的；

16.3 实例分组探索天气数据

- 读取数据，处理数据

```
fpath = "./datas/beijing_tianqi/beijing_tianqi_2018.csv"
df = pd.read_csv(fpath)
# 替换掉温度的后缀℃
df.loc[:, "bWendu"] = df["bWendu"].str.replace("℃", "").astype('int32')
df.loc[:, "yWendu"] = df["yWendu"].str.replace("℃", "").astype('int32')
df.head()
```

新增一列为月份

```
# 新增一列为月份
df['month'] = df['ymd'].str[:7]
df.head()
```

16.3.1 查看每个月的最高温度

- 查看每个月的最高温度

```
data = df.groupby('month')['bWendu'].max()
data


"""输出如下"""
month
2018-01     7
2018-02    12
2018-03    27
2018-04    30
2018-05    35
2018-06    38
2018-07    37
2018-08    36
2018-09    31
2018-10    25
2018-11    18
2018-12    10
Name: bWendu, dtype: int32
```

```
type(data)


"""输出如下"""
pandas.core.series.Series
```

```
data.plot()
```

16.3.2 查看每个月的最高温度、最低温度、平均空气质量指数

- 查看每个月的最高温度、最低温度、平均空气质量指数

```
df.head()
```

```
group_data = df.groupby('month').agg({"bWendu":np.max
                                      , "yWendu":np.min
                                      , "aqi":np.mean})
group_data
```

```
group_data.plot()
```

17 十六、Pandas_map、apply、applymap

- 数据转换函数对比：`map`、`apply`、`applymap`：
    1. `map`：只用于`Series`，实现每个值->值的映射；
    2. `apply`：用于`Series`实现每个值的处理，用于`Dataframe`实现某个轴的Series的处理；
    3. `applymap`：只能用于`DataFrame`，用于处理该`DataFrame`的每个元素；

17.1 map用于Series值的转换

- 实例：将股票代码英文转换成中文名字
    1. `Series.map(dict)`
    2. `Series.map(function)`

```
stocks = pd.read_excel('./datas/stocks/互联网公司股票.xlsx')
stocks.head()
```

```
array(['BIDU', 'BABA', 'IQ', 'JD'], dtype=object)# 查看有几个不同的公司
stocks['公司'].unique()



array(['BIDU', 'BABA', 'IQ', 'JD'], dtype=object)
```

映射公司名称为中文：

```
# 公司股票代码到中文的映射，注意这里是小写
dict_company_names = {
    "bidu": "百度",
    "baba": "阿里巴巴",
    "iq": "爱奇艺",
    "jd": "京东"
}
```

17.1.1 方法1：Series.map(dict)

- 使用字典映射：

```
stocks['中文名称1'] = stocks['公司'].str.lower().map(dict_company_names)
stocks.head()
```

17.1.2 方法2：Series.map(function)

- `function` 的参数是 `Series` 的每个元素的值

```
# 注意，这里的x是将Series中的每个元素进行 x.lower()
stocks['中文名称2'] = stocks['公司'].map(lambda x:
dict_company_names[x.lower()])
stocks.head()
```

17.2 apply用于Series和DataFrame的转换

1. `Series.apply(function)`，函数的参数是每个值
2. `DataFrame.apply(function)`，函数的参数是Series

17.2.1 Series.apply(function)

- `function` 的参数是 `Series` 的每个值

```
stocks['中文名称3'] = stocks['公司'].apply(lambda x :
dict_company_names[x.lower()])
stocks.head()
```

17.2.2 DataFrame.apply(function)

- `function` 的参数是对应轴的 `Series`

```
stocks["公司中文4"] = stocks.apply(
                    lambda x: dict_company_names[x['公司'].lower()]
                    , axis=1)
stocks.head()
```

注意这个代码：

1. `apply` 是在 `stocks` 这个 `DataFrame` 上调用；
2. `lambda` x的x是一个 `Series`，因为指定了 `axis` =1所以 `Seires` 的 `key` 是列名，可以用x['公司']获取

17.3 applymap用于DataFrame所有值的转换

- 转换 `DataFrame` 的所有值：

```
sub_df = stocks[['收盘', '开盘', '高', '低', '交易量']]
sub_df
```

将这些数字取整数，应用于所有元素

```
# 将这些数字取整数，应用于所有元素
sub_df.applymap(lambda x : int(x))
```

将这些数字保留两位小数，应用于所有元素

```
# 将这些数字保留两位小数，应用于所有元素
sub_df.applymap(lambda x : round(x,2))
```

直接修改原df的这几列

```
# 直接修改原df的这几列
stocks.loc[:, ['收盘', '开盘', '高', '低', '交易量']] =
sub_df.applymap(lambda x : int(x))
stocks.head()
```

18 十七、Pandas怎样对每个分组应用apply函数?

- 知识：`Pandas` 的 `GroupBy` 遵从 `split` 、 `apply` 、 `combine` 模式

  这里的 `split` 指的是 `pandas` 的 `groupby` ，我们自己实现 `apply` 函数， `apply` 返回的结果由 `pandas` 进行 `combine` 得到结果
- `GroupBy.apply(function)`
  1. `function` 的第一个参数是 `dataframe`
  2. `function` 的返回结果，可是 `dataframe` 、 `series` 、单个值，甚至和输入 `dataframe` 完全没关系
- 本次实例演示：
  1. 怎样对数值列按分组的归一化?
  2. 怎样取每个分组的TOPN数据?

18.1 实例1：怎样对数值列按分组的归一化?

- 将不同范围的数值列进行归一化，映射到[0,1]区间：
  1. 更容易做数据横向对比，比如价格字段是几百到几千，增幅字段是0到100
  2. 机器学习模型学的更快性能更好

  公式如下；

$$X_{normalized} = \frac{(X - X_{minimum})}{x_{maximum} - x_{minimum}} \tag{1}$$

18.1.1 演示：用户对电影评分的归一化

- 每个用户的评分不同，有的乐观派评分高，有的悲观派评分低，按用户做归一化

```
ratings = pd.read_csv(
    "./datas/movielens-1m/ratings.dat",
    sep="::",
    engine='python',
    names="UserID::MovieID::Rating::Timestamp".split("::")
)
ratings.head()
```

  实现按照用户ID分组，然后对其中一列归一化

```
# 实现按照用户ID分组，然后对其中一列归一化
def ratings_norm(df):
    """
    @param df: 每个用户分组的dataframe
    """
```

```
    min_value = df["Rating"].min()
    max_value = df["Rating"].max()
    df["Rating_norm"] = df["Rating"].apply(
        lambda x: (x-min_value)/(max_value-min_value))
    return df

ratings = ratings.groupby("UserID").apply(ratings_norm)

ratings

ratings[ratings["UserID"]==1].head()
```

可以看到UserID==1这个用户，Rating==3是他的最低分，是个乐观派，我们归一化到0分；

18.2 实例2：怎样取每个分组的TOPN数据？

- 获取2018年每个月温度最高的2天数据

```
fpath = "./datas/beijing_tianqi/beijing_tianqi_2018.csv"
df = pd.read_csv(fpath)
# 替换掉温度的后缀℃
df.loc[:, "bWendu"] = df["bWendu"].str.replace("℃", "").astype('int32')
df.loc[:, "yWendu"] = df["yWendu"].str.replace("℃", "").astype('int32')
# 新增一列为月份
df['month'] = df['ymd'].str[:7]
df.head()
```

```
df.shape[0]


"""输出如下"""
365
```

```
def get_wendu_topN(df,topn):
    """
    这里的df，是每个月份分组group的df
    """
    return df.sort_values(by='bWendu',ascending=False)[['ymd','bWendu']][:topn]


# 这里的这个topn    可以随便设置了
df.groupby('month').apply(get_wendu_topN,topn=2).head(10)
```

我们看到，`grouby` 的 `apply` 函数返回的 `dataframe`，其实和原来的 `dataframe` 其实可以完全不一样

19 十八、pandas 画图

- 导入包

```
import pandas as pd
import numpy as np

from matplotlib import pyplot as plt
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False
plt.rcParams["font.family"] = 'Arial Unicode MS'

import seaborn as sns
sns.set_palette("pastel", 8)
sns.set_palette("Blues_r", 8)
sns.set_palette("magma", 8)

```

19.1 基础图表

- 数据

```
df= pd.DataFrame(np.random.rand(8, 4), columns=['A','B','C','D'])
df
```

```
df1 = pd.DataFrame({'a': np.random.randn(1000) + 1,
                    'b': np.random.randn(1000),
                    'c': np.random.randn(1000) - 1},
                   columns=['a', 'b', 'c'])
df1.head()
```

19.1.1 柱状图

- 两种不同写法

```
df.plot.bar(alpha=0.5
            ,figsize=(12,8)
#             ,color='red'
           )
df.plot(kind='bar'
        ,alpha=0.5
        ,figsize=(12,8)
#         ,color='red'
       )
```

19.1.2 柱状图堆叠

- 两种不同写法

```
# 柱状图堆叠
df.plot.bar(stacked=True
            ,alpha=0.5
            ,figsize=(12,8)
#             ,color='red'
           )
df.plot(kind='bar'
        ,stacked=True
        ,alpha=1
        ,figsize=(12,8)
#         ,color='k'
       )
plt.title('柱状图堆叠')
```

19.1.3 堆叠柱状图

- 两种不同方法

```
## 堆叠柱状图
df.plot.barh(stacked=True
             ,alpha=0.5
             ,figsize=(12,8)
            )
df.plot(kind='barh'
        ,stacked=True
        ,alpha=1
        ,figsize=(12,8)
#         ,color='k'
       )
plt.title('堆叠柱状图')
```

19.1.4 面积图

- 两种不同写法

```
df.plot.area(stacked=False
             ,alpha = 0.7
             ,figsize=(12,8)
            )
df.plot(kind='area'
        ,stacked=False
        ,alpha=0.5
        ,figsize=(12,8)
#         ,color='k'
       )
plt.title('面积图')
```

19.1.5 堆叠面积图

- 两种不同写法

```
df.plot.area(stacked=True
             ,alpha = 0.7,
             figsize=(12,8)
            )
df.plot(kind='area'
        ,stacked=True
        ,alpha=0.5
        ,figsize=(12,8)
#         ,color='k'
       )
plt.title('堆叠面积图')
```

19.1.6 密度估计图

- 两种不同写法

```
## 密度估计图
df.plot.kde(alpha=0.5
            ,figsize=(12,8)
           )
df.plot(kind='kde'
        ,alpha=.5
        ,figsize=(12,8)
        ,color='k'
       )
plt.title('密度估计图')
```

19.1.7 直方图

- 堆叠直方图

```
## 堆叠、不堆叠的直方图
df1.plot.hist(stacked=True
              , bins=20
              ,figsize=(12,8)
             )# 堆叠

df1.plot(kind='hist'
         ,stacked=True
         , bins=20
         ,figsize=(12,8))
plt.title('堆叠直方图')
```

- 不堆叠直方图

```
## 不堆叠的直方图
df1.plot.hist(alpha=0.5
              ,figsize=(12,8)
#               ,color='k'
             ) # 不堆叠
df1.plot(kind='hist'
        ,alpha=1
        ,figsize=(12,8)
       )
plt.title('不堆叠的直方图')
```

19.1.8 箱线图

- 代码

```
# 箱线图
df1.plot.box(vert=False
             ,figsize=(12,8)
             ,color=dict(boxes='DarkGreen'
                   , whiskers='DarkOrange'
                   ,medians='DarkBlue'
                   , caps='Gray'
                  )
             , positions=[1, 4, 5])
plt.title('箱线图')
```

19.1.9 折线图

- 代码

```
df.plot(alpha=0.5
        ,figsize=(12,8)
#         ,color='k'
       )
plt.title('折线图')
```

19.1.10 散点图

- 代码

```
# 散点图
df1.plot.scatter(x='a'
                 ,y='b'
                 ,figsize=(12,8)
#                  ,c='c'
                 ,s=50
                )
plt.title('散点图')
```

19.1.11 六边形图

- 代码

```
# 六边形图
df1.plot.hexbin(x='a'
                ,y='b'
                ,gridsize=25
                ,figsize=(12,8)
               )
plt.title('六边形图')
```

19.2 多子图

- 代码

```
df1.plot(subplots=True
        ,layout=(2, 2)
        ,figsize=(15,8)
#         ,color='k'
       )

```

20 十九、pandas函数补充

20.1 pd.DataFrame.cumsum()

- 创建数据

```
import pandas as pd
fs = pd.DataFrame([[2.0, 1.0,3.0,5],
                   [3.0, 4.0,5.0,5],
                   [3.0, 4.0,5.0,5],
                   [1.0, 0.0,6.0,5]],
                  columns = list('ABCD')
                 )
fs
```

- 按行累加

```
# fs.cumsum(axis=0)
fs.cumsum(axis='index')
```

- 按列累加

```
# fs.cumsum(axis=1)
fs.cumsum(axis='columns')
```